Environmental Data Science Addenda

Jerry Davis, SFSU Institute for Geographic Information Science

2022-12-30

Contents

List of Figures

Chapter 1

Background, Goals and Data

1.1 Structure of the Addenda

These addenda may include:

- Case studies and extended methods, such as
  - air pollution studies
  - mapping methods
  - exploring other interpolation methods
  - models, e.g. a more extended seabird model
  - meadow research we're doing, in time and space domains
  - other imagery classification methods
- Guides
  - RMarkdown
  - Building code and data packages on GitHub

See specifics in the table of contents.

Note: some of these methods have made it into the main book, possibly in reduced form. The intention is in part to allow for a more extensive treatment in this addenda, and for the clearest and most succinct examples to make it to the next edition of the book, possibly replacing methods that might be moved here, as useful though not essential. And that's a good example of a non-succinct sentence, so it belongs here…

1.2 Software and Data

Some things covered in the main book, but repeated here for convenience.

```
## [1] "This book was produced in RStudio using R version 4.2.1 (2022-06-23 ucrt)"
```

For a start, you'll need to have R and RStudio installed, then you'll need to install various packages to support specific chapters and sections. No instructions are provided here, since you should know how to install packages in RStudio.

From time to time, you'll want to update your installed packages, and that usually happens when something doesn't work and maybe the dependencies of one package on another gets broken with a change in a package.

Fortunately, in the R world, especially at the main repository at CRAN, there's a lot of effort put into making sure packages work together, so usually there are no surprises if you're using the most current versions.

Once a package like `dplyr` is installed, you can access all of its functions and data by adding a library call, like …

```
library(dplyr)
```

… which you *will* want to include in your code, or to provide access to multiple libraries in the tidyverse, you can use `library(tidyverse)`. Alternatively, if you're only using maybe one function out of an installed package, you can call that function with the `::` separator, like `dplyr::select()`. This method has another advantage in avoiding problems with duplicate names – and for instance we'll generally call `dplyr::select()` this way.

1.2.1 Data

We'll be using data from various sources, including data on CRAN like the code packages above which you install the same way – so use `install.packages("palmerpenguins")`.

We've also created a repository on GitHub that includes data we've developed in the Institute for Geographic Information Science (iGISc) at SFSU, and you'll need to install that package a slightly different way.

GitHub packages require a bit more work on the user's part since we need to first install `remotes`[1], then use that to install the GitHub data package:

```
install.packages("remotes")
remotes::install_github("iGISc/igisci")
```

Then you can access it just like other built-in data by including:

```
library(igisci)
```

To see what's in it, you'll see the various datasets listed in:

```
data(package="igisci")
```

For instance, Figure 1.1 is a map of California counties using the CA_counties `sf` feature data. We'll be looking at the `sf` (Simple Features) package later in the Spatial section of the book, but seeing `library(sf)`, this is one place where you'd need to have installed another package, with `install.packages("sf")`.

[1] Note: you can also use `devtools` instead of `remotes` if you have that installed. They do the same thing; `remotes` is a subset of `devtools`. If you see a message about Rtools, you can ignore it since that is only needed for building tools from C++ and things like that.

```
library(tidyverse); library(igisci); library(sf)
ggplot(data=CA_counties) + geom_sf()
```

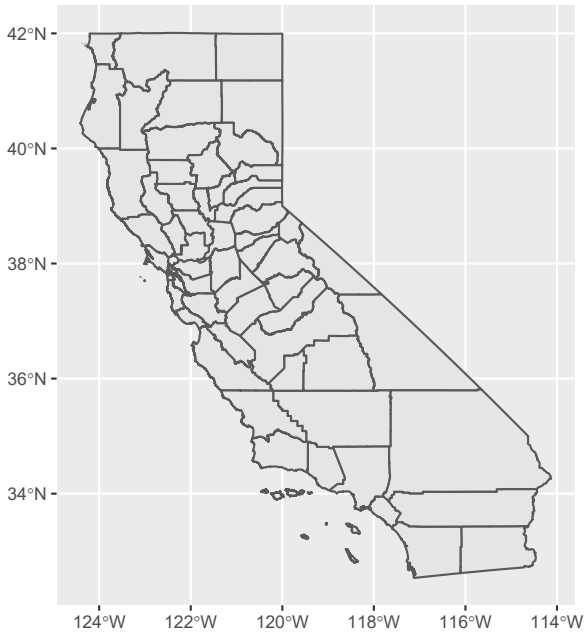


Figure 1.1 California counties simple features data in igisci package

The package datasets can be used directly as `sf` data or data frames. And similarly to functions, you can access the (previously installed) data set by prefacing with `igisci::` this way, without having to load the library. This might be useful in a one-off operation:

```
mean(igisci::sierraFeb$LATITUDE)
```

```
## [1] 38.3192
```

Raw data such as `.csv` files can also be read from the `extdata` folder that is installed on your computer when you install the package, using code such as:

```
csvPath <- system.file("extdata","TRI/TRI_1987_BaySites.csv", package="igisci")
TRI87 <- read_csv(csvPath)
```

or something similar for shapefiles, such as:

```
shpPath <- system.file("extdata","marbles/trails.shp", package="igisci")
trails <- st_read(shpPath)
```

And we'll find that including most of the above arcanity in a function will help. We'll look at functions later, but here's a function that we'll use a lot for setting up reading data from the extdata folder:

```
ex <- function(dta){system.file("extdata",dta,package="igisci")}
```

And this `ex()`function is needed so often that it's installed in the `igisci` package, so if you have `library(igisci)` in effect, you can just use it like this:

```
trails <- st_read(ex("marbles/trails.shp"))
```

But how do we see what's in the `extdata` folder? We can't use the `data()` function, so we would have to dig for the folder where the igisci package gets installed, which is buried pretty deeply in your user profile. So I wrote another function `exfiles()` that creates a data frame showing all of the files and the paths to use. In RStudio you could access it with `View(exfiles())` or we could use a datatable (you'll need to have installed "DT"). You can use the path using the `ex()` function with any function that needs it to read data, like `read.csv(ex('CA/CA_ClimateNormals.csv'))`, or just enter that `ex()` call in the console like `ex('CA/CA_ClimateNormals.csv')` to display where on your computer the installed data reside.

```
DT::datatable(exfiles(), options=list(scrollX=T), rownames=F)
```

Show 10 entries Search:

dir	file	path	type
BayArea	BayAreaCounties.shp	ex('BayArea/BayAreaCounties.shp')	shapefile
BayArea	BayAreaTracts.shp	ex('BayArea/BayAreaTracts.shp')	shapefile
BayArea	BayArea_hillsh.tif	ex('BayArea/BayArea_hillsh.tif')	TIFF
CA	CA_counties.shp	ex('CA/CA_counties.shp')	shapefile
CA	CAfreeways.shp	ex('CA/CAfreeways.shp')	shapefile
CA	ca_elev.tif	ex('CA/ca_elev.tif')	TIFF
CA	ca_elev_WGS84.tif	ex('CA/ca_elev_WGS84.tif')	TIFF
CA	ca_hillsh_WGS84.tif	ex('CA/ca_hillsh_WGS84.tif')	TIFF
CA	CA_ClimateNormals.csv	ex('CA/CA_ClimateNormals.csv')	CSV
CA	CA_MdInc.csv	ex('CA/CA_MdInc.csv')	CSV

Showing 1 to 10 of 94 entries Previous 1 2 3 4 5 … 10 Next

1.3 Acknowledgements

This compilation includes data and methods gathered by students in San Francisco State's GEOG 604/704 *Environmental Data Science* class, colleagues in the Department of Geography & Environment and NGOs such as PointBlue. And thanks again to Anna Studwell for the nice cover art: "Dandelion fluff – Ephemeral stalk sheds seeds to the universe".

Chapter 2

Air pollutant emissions trends (EPA)

The National Emissions Inventory (NEI) program of the US EPA is a detailed estimate of air emissions of criteria and hazardous air pollutants from a wide variety of air emissions sources. The inventory is released every three years based primarily upon data provided by state, local, and tribal air resources agencies for pollution sources they monitor, supplemented by data developed by the US EPA.

Air pollutant data were downloaded from https://www.epa.gov/air-emissions-inventories/air-pollutant-emissions-trends-data and provided in the igisci package.

Processing these data to create a `free_y` faceted graph (2.1) employs several data transformation methods we've looked at, and some we haven't:

- `summarize_all` gets means of all variables, though since we're just using the totals, this just causes worksheets with multiple `Total` rows to merge into one; they're actually the same value
- `pivot_longer` is used twice, first to create columns for each pollutant, so the columns can be binded together, then later to create a facet graph where each pollutant becomes a parameter factor
- `bind_cols` to combine a series of data frames with each having the same years (1990:2016) when data for all parameters were collected
- A fix for dealing data not all being read in as numeric, needed for an entry error for the NH3 data, was developed by first reading in the `Source Category` column, then the yearly data values, setting `col_types="numeric"`, then binding the columns

```
library(tidyverse); library(readxl); library(igisci)
dtaPath <- ex("airquality/Pollution by type US 1970 to 2016.xlsx")
YearColumn <- readxl::read_xlsx(dtaPath, sheet = "SO2", skip=2) %>%
  pivot_longer(cols=`1990`:`2016`, names_to="Year") %>%
  dplyr::select(Year) %>% mutate(Year=as.numeric(Year))
YearCol <- as.data.frame(unique(YearColumn$Year))
names(YearCol) = "Year"

getPollutant <- function(pollutant){
  thedata <- readxl::read_xlsx(dtaPath,sheet=pollutant,skip=2,
                               col_types="numeric") %>%
     dplyr::select(-`Source Category`)
  rowheaders <- readxl::read_xlsx(dtaPath,sheet=pollutant,skip=2) %>%
     dplyr::select(`Source Category`)
  bind_cols(rowheaders,thedata) %>%
     dplyr::select(`Source Category`,`1990`:`2016`) %>%
     filter(`Source Category`=="Total") %>%
```

```
    group_by(`Source Category`) %>%
    summarize_all(list(mean)) %>%
    pivot_longer(cols=`1990`:`2016`,names_to="Year",values_to=pollutant) %>%
    dplyr::select(pollutant)
}
SO2  <- getPollutant("SO2")
PM25 <- getPollutant("PM25Primary") %>% rename(PM25=PM25Primary)
PM10 <- getPollutant("PM10Primary") %>% rename(PM10=PM10Primary)
NOX  <- getPollutant("NOX")
CO   <- getPollutant("CO")
VOC  <- getPollutant("VOC")
NH3  <- getPollutant("NH3")
pollutants <- bind_cols(YearCol,CO,NOX,SO2,PM25,PM10,VOC,NH3)#

pollutant_long <- pollutants %>%
  pivot_longer(cols = CO:NH3, names_to="parameter", values_to="value")
p <- ggplot(data = pollutant_long, aes(x=Year, y=value)) + geom_line()
p + facet_grid(parameter ~ ., scales = "free_y")
```

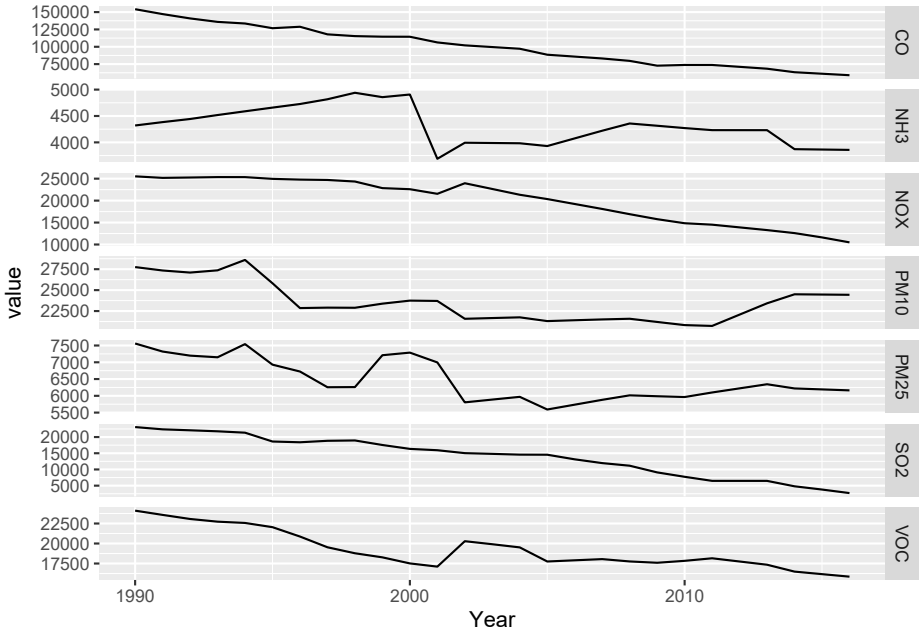


Figure 2.1 Facet graph of air pollutants in the US, 1990-2016

Chapter 3

Distance from census tract centroid to closest TRI site

In the Spatial Analysis chapter section on Distance and specifically Distance to Closest Feature, we looked at point to closest point distances. Here's another example of this that attempts to find the output of closest TRI facility to a census tract where we might compare health or socioeconomic variables to that value. The `igisci::BayAreaTracts` data has census data from 2010 by census tracts in the SF Bay area. In addition to the distance functions, the code below includes a few useful `sf` functions applied to that data.

```
library(igisci); library(sf); library(tidyverse)
TRI_CA <- read_csv(ex("TRI/TRI_2017_CA.csv"))
TRI_BySite <- TRI_CA %>%
  mutate(all_air = `5.1_FUGITIVE_AIR` + `5.2_STACK_AIR`) %>%
  mutate(carcin_release = all_air * (CARCINOGEN == "YES")) %>%
  group_by(TRI_FACILITY_ID) %>%
  summarise(
    count = n(),
    air_releases = sum(all_air, na.rm = TRUE),
    fugitive_air = sum(`5.1_FUGITIVE_AIR`, na.rm = TRUE),
    stack_air = sum(`5.2_STACK_AIR`, na.rm = TRUE),
    FACILITY_NAME = first(FACILITY_NAME),
    COUNTY = first(COUNTY),
    LATITUDE = first(LATITUDE),
    LONGITUDE = first(LONGITUDE))
co <- st_read(ex("BayArea/BayAreaCounties.shp"))
freeways <- st_read(ex("CA/CAfreeways.shp"))
censusBayArea <- st_make_valid(st_read(ex("BayArea/BayAreaTracts.shp")))
incomeByTract <- read_csv(ex("CA/CA_MdInc.csv")) %>%
  dplyr::select(trID, HHinc2016) %>%
  mutate(HHinc2016 = as.numeric(str_c(HHinc2016)),
         joinid = str_c("0", trID))
censusBayArea <- censusBayArea %>%
  mutate(whitepct = WHITE / POP2010 * 100) %>%
  mutate(People_of_Color_pct = 100 - whitepct) %>%
  left_join(incomeByTract, by = c("FIPS" = "joinid"))
censusCentroids <- st_centroid(censusBayArea)
census <- censusBayArea
bnd <- st_bbox(co)
```

```
TRIdata <- TRI_BySite %>%
  filter(air_releases > 0) %>%
  filter((LONGITUDE < bnd[3]) & (LONGITUDE > bnd[1]) & (LATITUDE < bnd[4]) & (LATITUDE > bnd[2]))
BA_counties <- co %>% dplyr::select(NAME, CNTY_FIPS) %>% st_set_geometry(NULL)
TRI_sp <- st_as_sf(TRIdata, coords = c("LONGITUDE", "LATITUDE"), crs=4326) %>%
  st_join(censusBayArea) %>%
  filter(CNTY_FI %in% BA_counties$CNTY_FIPS)
TRI2join <- TRI_sp %>%
  st_set_geometry(NULL) %>%
  rowid_to_column("TRI_ID")
```

```
ggplot() +
  geom_sf(data = co, fill = NA) +
  geom_sf(data=TRI_sp) +
  coord_sf(xlim = c(bnd[1], bnd[3]), ylim = c(bnd[2], bnd[4])) +
  labs(x='',y='')
```

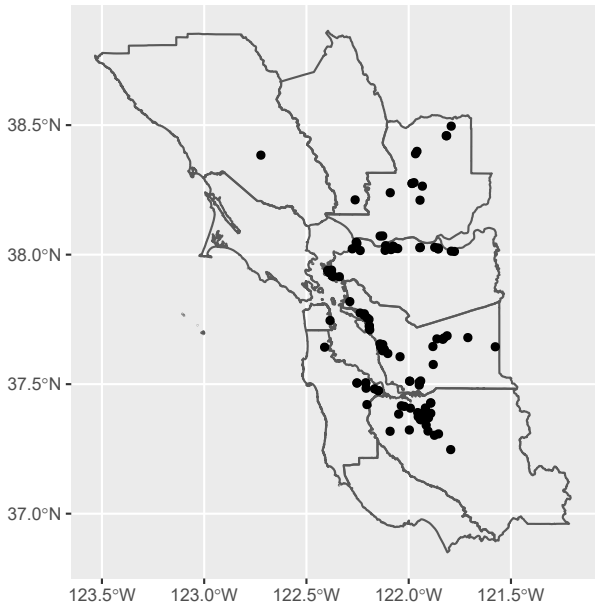


Figure 3.1 EPA Toxic Release Inventory (TRI) facilities 2017

Asthma model related to distance to nearest TRI facility.

```
# CDC data
CDCdata <- read_csv(ex("TRI/CDC_health_data_by_Tract_CA_2018_release.csv")) %>%
  mutate(
    lon = as.numeric(str_sub(Geolocation, 17, 30)),
    lat = as.numeric(str_sub(Geolocation, 2, 14)),
    CDC_CNTY_FIPS = str_sub(TractFIPS, 2, 4)) %>%
  filter(CDC_CNTY_FIPS %in% BA_counties$CNTY_FIPS)
CDCbay <- st_as_sf(CDCdata, coords = c('lon', 'lat'), crs=4326)

# D to TRI:  create vector of index of nearest TRI facility to each CDC (tract centroid) point
nearest_TRI_ids <- st_nearest_feature(CDCbay, TRI_sp)
```

```
# get locations of each NEAREST TRI facility only
near_TRI_loc <- TRI_sp$geometry[nearest_TRI_ids]

CDCbay <- CDCbay %>%
  mutate(d2TRI = st_distance(CDCbay, near_TRI_loc, by_element=TRUE),
         d2TRI = units::drop_units(d2TRI),
         nearest_TRI = nearest_TRI_ids) %>%
  left_join(BA_counties, by = c("CDC_CNTY_FIPS" = "CNTY_FIPS"))

CDCTRIbay <- left_join(CDCbay, TRI2join, by = c("nearest_TRI" = "TRI_ID")) %>%
  filter(d2TRI > 0) # %>%
```

```
CDCTRIbay %>%
  ggplot(aes(air_releases, CASTHMA_CrudePrev)) +
  geom_point(aes(color = NAME)) + geom_smooth(method="lm", se=FALSE, color = "black") +
  labs(x = "Air releases at nearest TRI facility",
       y = "Asthma Prevalence CDC Model, Adults >= 18, 2016")
```

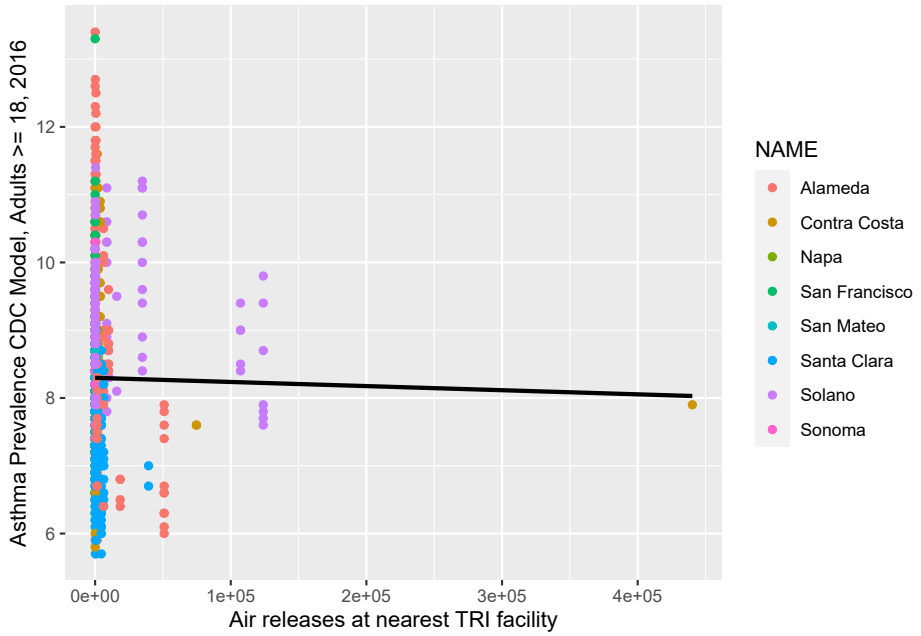


Figure 3.2 Asthma prevalence related to nearest TRI total releases

This is an admittedly crude analysis, lumping all releases together, and so it's not surprising that there's no clear relationship over the whole Bay Area. There's quite a lot you could explore in the TRI data, and you might want to focus on one county, specific toxins, or probably aggregate releases at the total output in the area. And analyzing health data is very challenging given the wide range of factors influencing patterns. You might also consider looking at earlier TRI data (the `igisci` extdata `"TRI"` folder has it back to 1987, for California.)

Chapter 4

Exploring terrain and water samples in a karst area

Sinking Cove, Tennessee is a karst valley system carved into the Cumberland Plateau, a nice place to see the use of a hillshade raster created from a digital elevation model using raster or terra functions for slope, aspect, and hillshade. We'll look at this area and some results from a hydrologic and geochemical study (Davis and Brook 1993):

Figure 4.1 Cave Cove Cave Entrance

We'll start with a regional view using a hillshade and terrain shading:

```
tmap_mode("plot")
DEM <- rast(ex("SinkingCove/DEM_SinkingCoveUTM.tif"))
slope <- terrain(DEM, v='slope')
aspect <- terrain(DEM, v='aspect')
hillsh <- shade(slope/180*pi, aspect/180*pi, angle=40, direction=330)
# Need to crop a bit since grid north != true north
bbox0 <- st_bbox(DEM)
xrange <- bbox0$xmax - bbox0$xmin
yrange <- bbox0$ymax - bbox0$ymin
bbox1 <- bbox0
crop <- 0.05
bbox1[1] <- bbox0[1] + crop * xrange # xmin
bbox1[3] <- bbox0[3] - crop * xrange # xmax
bbox1[2] <- bbox0[2] + crop * yrange # ymin
bbox1[4] <- bbox0[4] - crop * yrange # ymax
```

Figure 4.2 Sinking Cove Cave

Figure 4.3 Sinking Cove Cave spring

```
bboxPoly <- bbox1 %>% st_as_sfc() # makes a polygon
#
tm_shape(hillsh, bbox=bboxPoly) +
  tm_raster(palette="-Greys",legend.show=F,n=20) +
  tm_shape(DEM) +
  tm_raster(palette=terrain.colors(24), alpha=0.5) +
  tm_graticules(lines=F)
```

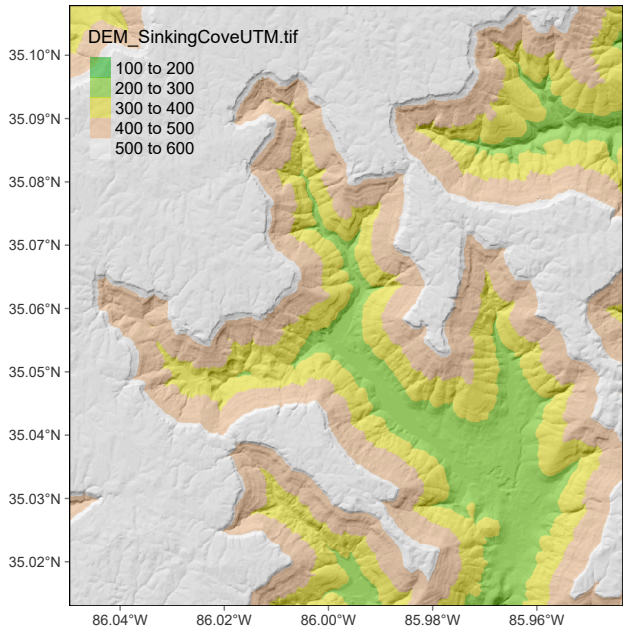


Figure 4.4 Sinking Cove Area map, using terra rasters in tmap

Then zoom into Upper Sinking Cove to look at some water sample results:

```
library(igisci)
library(sf); library(tidyverse); library(readxl); library(tmap)
wChemData <- read_excel(ex("SinkingCove/SinkingCoveWaterChem.xlsx")) %>%
  mutate(siteLoc = str_sub(Site,start=1L, end=1L))
wChemTrunk <- wChemData %>% filter(siteLoc == "T") %>% mutate(siteType = "trunk")
wChemDrip <- wChemData %>% filter(siteLoc %in% c("D","S")) %>% mutate(siteType = "dripwater")
wChemTrib <- wChemData %>% filter(siteLoc %in% c("B", "F", "K", "W", "P")) %>%
  mutate(siteType = "tributary")
wChemData <- bind_rows(wChemTrunk, wChemDrip, wChemTrib)
sites <- read_csv(ex("SinkingCove/SinkingCoveSites.csv"))
wChem <- wChemData %>%
  left_join(sites, by = c("Site" = "site")) %>%
  st_as_sf(coords = c("longitude", "latitude"), crs = 4326)
library(terra)
tmap_mode("plot")
DEM <- rast(ex("SinkingCove/DEM_SinkingCoveUTM.tif"))
slope <- terrain(DEM, v='slope')
aspect <- terrain(DEM, v='aspect')
hillsh <- shade(slope/180*pi, aspect/180*pi, 40, 330)
bounds <- st_bbox(wChem)
```

```
xrange <- bounds$xmax - bounds$xmin
yrange <- bounds$ymax - bounds$ymin
xMIN <- as.numeric(bounds$xmin - xrange/10)
xMAX <- as.numeric(bounds$xmax + xrange/10)
yMIN <- as.numeric(bounds$ymin - yrange/10)
yMAX <- as.numeric(bounds$ymax + yrange/10)
newbounds <- st_bbox(c(xmin=xMIN, xmax=xMAX, ymin=yMIN, ymax=yMAX), crs= st_crs(4326))
tm_shape(hillsh,bbox=newbounds) +
  tm_raster(palette="-Greys",legend.show=F,n=20) +
  tm_shape(DEM) + tm_raster(palette=terrain.colors(24), alpha=0.5,legend.show=F) +
  tm_shape(wChem) + tm_symbols(size="TH", col="Lithology", scale=2, shape="siteType") +
  tm_layout(legend.position = c("left", "bottom")) +
  tm_graticules(lines=F)
```

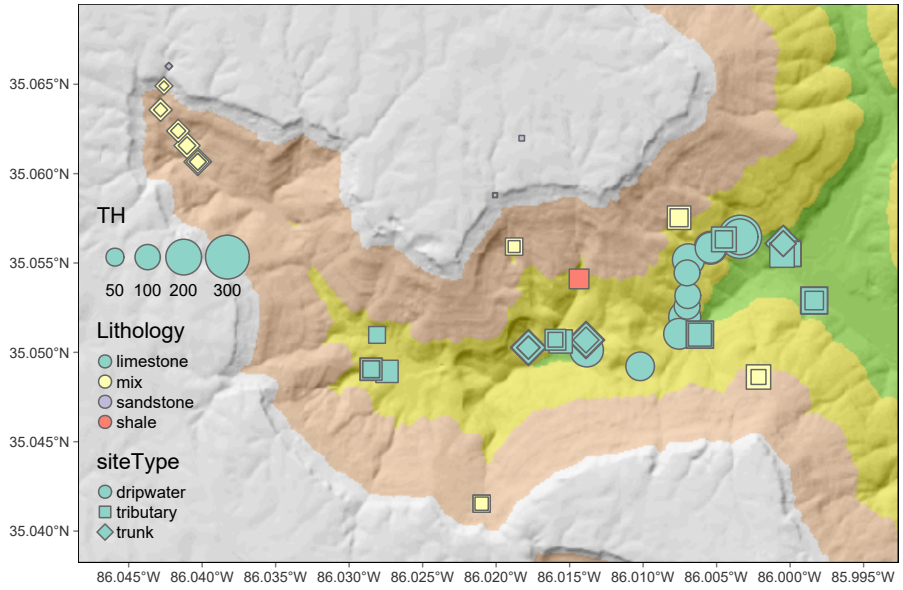


Figure 4.5 Upper Sinking Cove water sample results

An interactive map view of Upper Sinking Cove:

```
tmap_mode("view")
bounds <- st_bbox()
wChem2map <- filter(wChem, Month == 8)
minVal <- min(wChem2map$TH); maxVal <- max(wChem2map$TH)
tm_basemap(leaflet::providers$Esri.WorldTopoMap) +
  tm_shape(wChem2map) + tm_symbols(col="siteType", size="TH", scale=2) +
  tm_layout(title=paste("Total Hardness ",as.character(minVal),"-",as.character(maxVal)," mg/L", sep=""))
```

```
tm_basemap(leaflet::providers$Esri.WorldTopoMap) +
  tm_shape(wChem2map) + tm_symbols(col="Lithology", size="TH", scale=2)
```

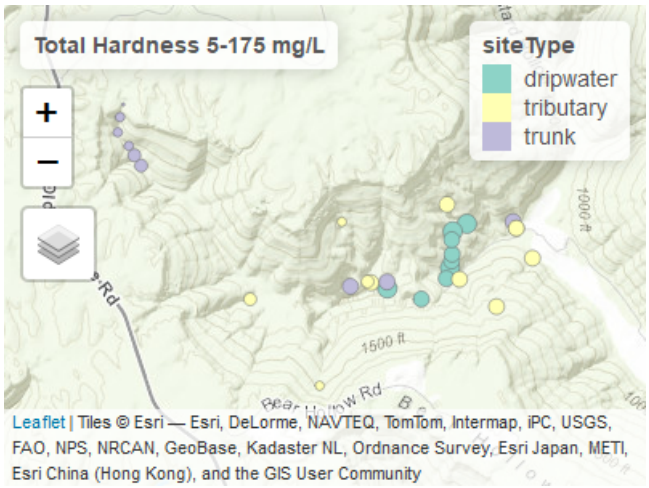


Figure 4.6 Upper Sinking Cove interactive map

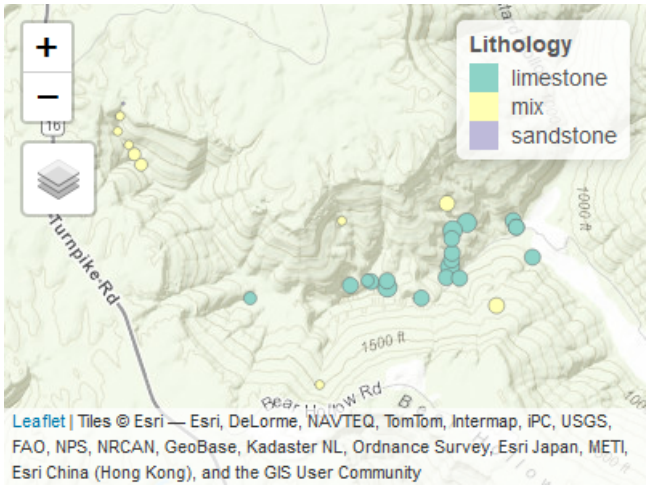


Figure 4.7 Upper Sinking Cove interactive map

```
tmap_mode("plot")
```

Chapter 5

Other Spatial Methods & Data Experiments

5.1 Plotting filtered map data using base plot, terra::plotRGB and SpatVector data

In the Spatial Analysis chapter, we've mainly used tmap, but we might also want to see how we'd plot the same thing in the base plot system, which also support's terra's SpatVector data. So in this brief example, we'll create a SpatVector after a data filter, then plot in the base plot system.

```
library(tidyverse); library(sf); library(igisci)
sierraFebpts <- st_as_sf(read_csv(ex("sierra/sierraFeb.csv")),
                         coords = c("LONGITUDE", "LATITUDE"), remove=FALSE, crs=4326)
```

We might also look at how we get information about out data to decide what we might want to plot on the map. Since our goal is to filter for a range of elevations and latitudes, we might look at a couple of histograms, and we'll use a base R plot's parameter setting (`par`) to create a side by side plot (1 row, 2 columns).

```
par(mfrow=c(1,2))  # lets base plot functions create 2 adjacent plots (1 row, 2 columns)
hist(sierraFebpts$ELEVATION)
hist(sierraFebpts$LATITUDE)
```

```
par(mfrow=c(1,1)) # need to reset to defaults
```

Now let's continue with that information where we've gotten an idea about what kinds of elevations occur at the stations, and make a map that shows all stations > 2000 m elevation.

We'll briefly use `terra::plotRGB` to add a basemap to a static map. *Warning: the `get_tiles` function goes online to get the basemap data, so if you don't have a good internet connection or the site goes down, this may fail.* We'll also modify the station names and set the text label position as `pos=3`, which means centered on top (see `?graphics::text`).

```
library(terra); library(maptiles)
sierraFebpts2000 <- sierraFebpts %>%
  filter(ELEVATION >= 2000)
```

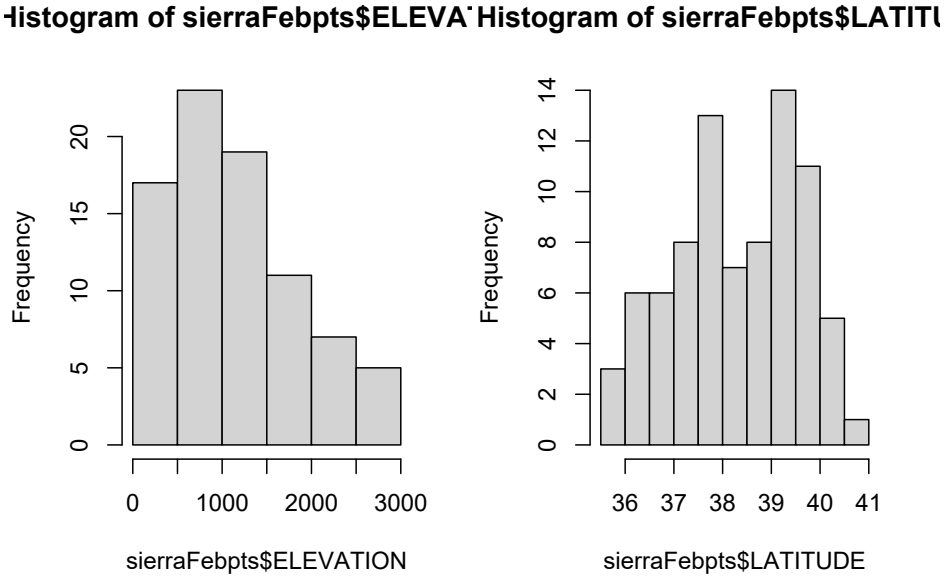


Figure 5.1 Histograms of Sierra station elevations and latitudes

```
sierraBase <- get_tiles(sierraFebpts2000)
plotRGB(sierraBase)
pts <- vect(sierraFebpts2000)
points(pts, col="red")
text(pts, labels=str_sub(pts$STATION_NAME, 1, str_locate(pts$STATION_NAME, ",")-1), cex = 0.7, pos=3)
```

Before we continue, let's look at some of the parameters just used for sizing and placing text labels, and figure out what they do:

```
text(pts, labels=str_sub(pts$STATION_NAME, 1, str_locate(pts$STATION_NAME, ",")-1), cex = 0.7,
pos=3)
```

- What does `cex` do? (Use `?par` or `?text` then search within that help)
- What does `pos` do? (Use `?text`)
- What did the stringr methods do?

Tweak each of the above to see the effect on the map.

Maybe there's a way with `plotRGB`, but it's limited in not apparently having a way to provide a *graticule* of latitude and longitude, so its use as a map is limited, and there are generally better options in tmap.

Figure 5.2 Plotting filtered data (ELEVATION >= 2000)

Chapter 6

PointBlue Penguin Study

Currently, this is the same as in the book, but will be expanded.

6.0.1 Interactive mapping of individual penguins abstracted from a big dataset

In a study of spatial foraging patterns by Adélie penguins (*Pygoscelis adeliae*), Ballard et al. (2019) collected data over five austral summer seasons (parts of December and January in 2005-06, 2006-07, 2007-08, 2008-09, 2012-13) period using SPLASH tags that combine Argos satellite tracking with a time-depth recorder, with 11,192 observations. The 24 SPLASH tags were re-used over the period of the project with a total of 162 individual breeding penguins, each active over a single season.

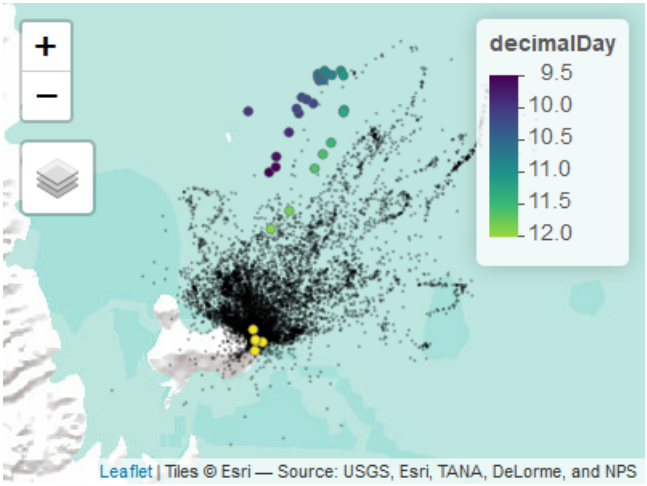
Our code takes advantage of the abstraction capabilities of base R and `dplyr` to select an individual penguin from the large data set and prepare its variables for useful display. Then the interactive mode of tmap works well for visualizing the penguin data – both the cloud of all observations and the focused individual – since we can zoom in to see the locations in context, and basemaps can be chosen. The `tmap` code colors the individual penguin's locations based on a decimal day derived from day and time. While interactive view is more limited in options, it does at least support a legend and title (Figure **??**).

```
set.seed(28)
```

```
library(sf); library(tmap); library(tidyverse); library(igisci)
sat_table <- read_csv(ex("penguins/sat_table_splash.csv"))
obs <- st_as_sf(filter(sat_table,include==1), coords=c("lon","lat"), crs=4326)
uniq_ids <- unique(obs$uniq_id) # uniq_id identifies an individual penguin
onebird <- obs %>%
  filter(uniq_id==uniq_ids[sample.int(length(uniq_ids), size=1)]) %>%
  mutate(decimalDay = as.numeric(day) + as.numeric(hour)/24)
```

```
tmap_mode = "view"
tmap_options(basemaps=c(Terrain = "Esri.WorldTerrain",
                        Imagery = "Esri.WorldImagery",
                        NatGeo = "Esri.NatGeoWorldMap",
                        Topo="OpenTopoMap",
                        Ortho="GeoportailFrance.orthos"))
penguinMap <- tm_basemap() +
  tm_shape(obs) + tm_symbols(col="black", border.lwd=0, size=0.01, alpha=0.33) +
  tm_shape(onebird) + tm_symbols(col="decimalDay",
  palette="viridis", style="cont",n=8,size=0.6) +
```

```
  tm_legend() +
  tm_layout()
tmap_leaflet(penguinMap)
```



Chapter 7

Seabird Model

In this case study, we'll expand upon a **poisson** family glm applied to seabird counts discussed in the modeling chapter. The Applied California Current Ecosystem Study (ACCESS) https://farallones.noaa.gov/science/access.html supports marine wildlife conservation in northern and central California, including the Greater Farallones off the Golden Gate and San Francisco (*Applied California Current Ecosystem Studies* (n.d.), Studwell et al. (2017)).

Figure 7.1 By Caleb Putnam - Black-footed Albatross, 20 miles offshore of Newport, OR, 16 July 2013, CC BY-SA 2.0, https://commons.wikimedia.org/w/index.php?curid=74693160

```
library(igisci); library(sf); library(tidyverse); library(tmap)
library(maptiles); library(readxl); library(DT)
Sanctuaries <- st_read(ex("SFmarine/Sanctuaries.shp"))
transectsXLS <- read_xls(ex("SFmarine/TransectsData.xls"))
transects <- st_transform(st_as_sf(transectsXLS, coords=c("midlon","midlat"), crs=4326), crs=st_crs(Sanctuaries))
cordell_bank <- st_read(ex("SFmarine/cordell_bank.shp"))
```

```
isobath_200 <- st_read(ex("SFmarine/isobath_200.shp"))
mainland <- st_read(ex("SFmarine/mainland.shp"))
sefi <- st_read(ex("SFmarine/sefi.shp"))
```

```
library(tmap); library(maptiles)
tmap_mode("plot")
oceanBase <- get_tiles(transects, provider="Esri.NatGeoWorldMap")
transJul <- transects %>% filter(month==7)
transJulnonegTS <- transJul %>% filter(avg_tem>0 & avg_sal>0)
transJulnonegTSF <- transJulnonegTS %>% filter(avg_fluo>0)
Jul_bfal <- tm_shape(oceanBase) + tm_rgb() +
  tm_shape(transJul) + tm_symbols(col="bfal", size="bfal")
Jul_all <- tm_shape(oceanBase) + tm_rgb() +
  tm_shape(transJul) + tm_dots(col="avg_fluo", size="avg_fluo")
Jul_TS <- tm_shape(oceanBase) + tm_rgb() +
  tm_shape(transJulnonegTS) + tm_dots(col="avg_fluo", size="avg_fluo")
Jul_TSF <- tm_shape(oceanBase) + tm_rgb() +
  tm_shape(transJulnonegTSF) + tm_dots(col="avg_fluo", size="avg_fluo")

tmap_arrange(Jul_bfal, Jul_all, Jul_TS, Jul_TSF)
```

The transects data have 45 variables including:

- date and time variables
- seawater measurements of temperature, salinity and fluorescence
- ocean climate indices
- distance to land, islands, the 200m isobath, and Cordell Bank
- depth
- oceanic/atmospheric conditions (sea state, visibility, beaufort index, cloudiness, upwelling)
- counts of seabirds:
  - black-footed albatross (bfal)
  - northern fulmar (nofu)
  - pink-footed shearwater (pfsh)
  - red phalarope (reph)
  - red-necked phalarope (rnph)
  - sooty shearwater (sosh)

More specifics are in the metadata:

```
transects_metadata <- read_excel(ex("SFmarine/Transects_Metadata.xls"))
DT::datatable(transects_metadata[1,1])
```

Show 10 entries Search:

	Data used in Studwell et al. 2017. "Modeling nonresident seabird foraging distributions to inform ocean zoning in central California." PLoS ONE.https://doi.org/10.1371/journal.pone.0169517
1	For clarification, questions, or additional data please direct requests to jjahncke@pointblue.org or marinedirector@pointblue.org. Point Blue Conservation Science data included here available for sharing, within the context of a data sharing agreement (http://www.pointblue.org/datasharing)

Showing 1 to 1 of 1 entries Previous 1 Next

```
DT::datatable(read_excel(ex("SFmarine/Transects_Metadata.xls"),skip=3))
```

Show 10 entries Search:

	Variables included	...2
1	year	Year data were collected
2	month	Month data were collected
3	obsdate	Local observation date (YYMMDD)
4	time	Local time of bin start (HHMMSS)
5	endtime	Local time of bin end (HHMMSS)
6	midlat	Middle latitude of bin (decimal degrees)
7	midlon	Middle longitude of bin (decimal degrees)
8	avg_tem	Average surface temperature (øC) using interpolated thermosalinograph and CTD surfaces
9	avg_sal	Average surface salinity (mg/m^3) using interpolated thermosalinograph and CTD surfaces
10	avg_fluo	Average surface fluorescence (rfu) using interpolated thermosalinograph and CTD surfaces

Showing 1 to 10 of 44 entries Previous 1 2 3 4 5 Next

7.1 Goals and basic methods of the analysis

Our general goal, inspired by Studwell et al. (2017), is to look at what variables influence the counts of specific seabirds and develop predictive models and maps illustrating optimal conditions for those seabirds based on the model. We're expecting that some of the various measurements and conditions could be good predictors, though recognizing that no model is perfect and there are going to be many other factors we can't account for.

Our basic method will include:

- Gathering data collected on the cruises as well as GIS and model-derived measurements (e.g. depth, distance, climate models). *This has been done and all variables we'll use are included in the transects shapefile.*

- Explore the data using maps, graphs and tables, filtering for a time frame and complete data.
- Select a species, timeframe, and selection of explanatory variables.
- Use `glm` to model the species counts responding to the explanatory variables, using the poisson family.
- Map the results.

7.2 Exploratory data analysis

We'll start by looking at a summary of the bird counts, stored as a series of variables from `bfal` (black-footed albatross) to `sosh` (sooty shearwater):

```
transectBirdcounts <- transects %>%
  dplyr::select(bfal:sosh)
summary(transectBirdcounts)
```

```
##       bfal              nofu               pfsh              reph
##  Min.   : 0.0000   Min.   :  0.0000   Min.   : 0.0000   Min.   : 0.0000
##  1st Qu.: 0.0000   1st Qu.:  0.0000   1st Qu.: 0.0000   1st Qu.: 0.0000
##  Median : 0.0000   Median :  0.0000   Median : 0.0000   Median : 0.0000
##  Mean   : 0.1169   Mean   :  0.2001   Mean   : 0.1883   Mean   : 0.1638
##  3rd Qu.: 0.0000   3rd Qu.:  0.0000   3rd Qu.: 0.0000   3rd Qu.: 0.0000
##  Max.   :20.0000   Max.   :109.0000   Max.   :50.0000   Max.   :70.0000
##       rnph              sosh                    geometry
##  Min.   : 0.0000   Min.   :   0.000   POINT        :4073
##  1st Qu.: 0.0000   1st Qu.:   0.000   epsg:6414    :   0
##  Median : 0.0000   Median :   0.000   +proj=aea ...:   0
##  Mean   : 0.3351   Mean   :   5.737
##  3rd Qu.: 0.0000   3rd Qu.:   0.000
##  Max.   :86.0000   Max.   :1405.000
```

We'll use the July data from multiple years (in the modeling chapter we just looked at July 2006) to visualize the spatial patterns of black-footed albatross, which we can see from the above has the lowest counts, displayed in both color and size in order to better see the larger counts. We'll make similar maps of measurements of temperature, salinity and fluorescence from the transect cruises. Note that records with values of zero for any of these 3 measures are excluded, since these represent non-measurements.

```
tmap_mode("plot")
oceanBase <- get_tiles(transects, provider="Esri.NatGeoWorldMap")
transJul <- transects %>% filter(month==7 & !is.na(avg_tem) & !is.na(avg_sal) & !is.na(avg_fluo))
Jul_bfal <- tm_shape(oceanBase) + tm_rgb() +
  tm_shape(transJul) + tm_symbols(col="bfal", size="bfal")
Jul_tem <- tm_shape(oceanBase) + tm_rgb() +
  tm_shape(transJul) + tm_dots(col="avg_tem", size="avg_tem")
Jul_sal <- tm_shape(oceanBase) + tm_rgb() +
  tm_shape(transJul) + tm_dots(col="avg_sal", size="avg_sal")
Jul_fluo <- tm_shape(oceanBase) + tm_rgb() +
  tm_shape(transJul) + tm_dots(col="avg_fluo", size="avg_fluo")
tmap_arrange(Jul_bfal, Jul_tem, Jul_sal, Jul_fluo)
```

7.2.1 Identifying the appropriate model using variance and mean comparisons

The poisson model uses an assumption that the mean and variance of the response variable is equal, but a quick check shows that this is not really met, and that since the variance is greater than the mean, our data

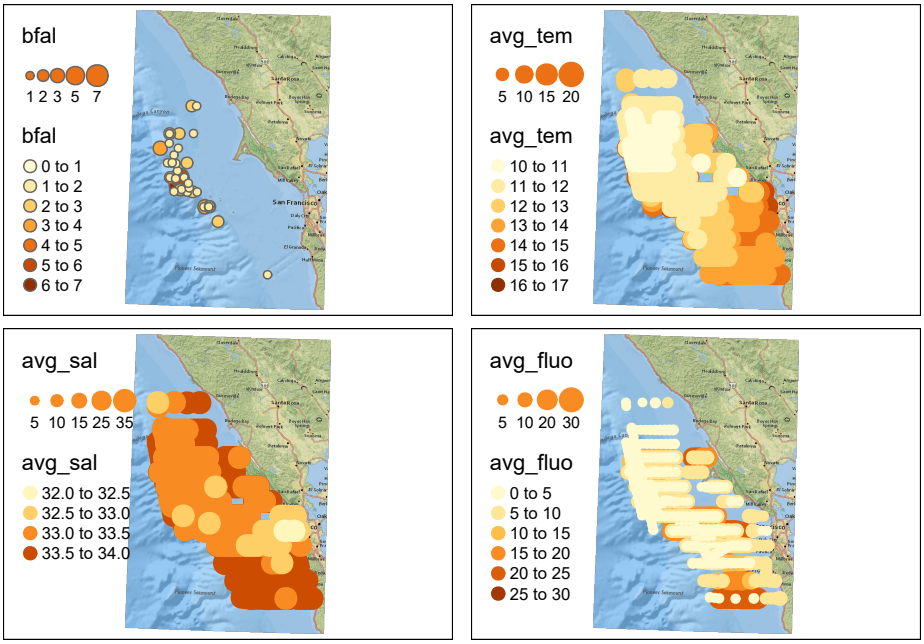


Figure 7.2 July black footed albatross, temperature, salinity, fluorescence

may be *overdispersed*, which in practice is very common.

```
mean(transJul$bfal)
```

```
## [1] 0.08034611
```

```
var(transJul$bfal)
```

```
## [1] 0.1903187
```

7.3 Model black-footed albatross counts for July using a poisson-family glm

Prior studies have suggested that temperature, salinity, fluorescence, depth and various distances might be good explanatory variables to use to look at spatial patterns, so we'll use these in the model. (Variables such as climate and oceanic conditions such as upwelling might be used in a temporal analysis, but are constant for this July analysis where we're modeling variables with spatial patterns.)

```
summary(glm(bfal~avg_tem+avg_sal+avg_fluo+avg_dep+dist_land+dist_isla+dist_200m+dist_cord+year, data=transJul,fam
```

```
##
## Call:
## glm(formula = bfal ~ avg_tem + avg_sal + avg_fluo + avg_dep +
##     dist_land + dist_isla + dist_200m + dist_cord + year, family = poisson,
##     data = transJul)
##
```

```
## Deviance Residuals:
##     Min       1Q   Median       3Q      Max
## -1.2692  -0.4233  -0.1760  -0.0382   5.5544
##
## Coefficients:
##               Estimate Std. Error z value Pr(>|z|)
## (Intercept) -6.335e+02  1.955e+02  -3.241  0.00119 **
## avg_tem      1.834e-01  1.217e-01   1.506  0.13199
## avg_sal      1.893e+00  7.643e-01   2.476  0.01327 *
## avg_fluo    -1.803e-01  6.458e-02  -2.792  0.00524 **
## avg_dep     -2.469e-03  5.085e-04  -4.854 1.21e-06 ***
## dist_land   -7.343e-05  3.436e-05  -2.137  0.03261 *
## dist_isla    8.140e-06  1.129e-05   0.721  0.47107
## dist_200m   -3.444e-04  6.478e-05  -5.317 1.05e-07 ***
## dist_cord   -5.801e-06  9.461e-06  -0.613  0.53975
## year         2.836e-01  9.088e-02   3.121  0.00180 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for poisson family taken to be 1)
##
##     Null deviance: 412.03  on 808  degrees of freedom
## Residual deviance: 295.41  on 799  degrees of freedom
## AIC: 408.82
##
## Number of Fisher Scoring iterations: 8
```

We can see in the model coefficients table several predictive variables that appear to be significant:

- avg_sal
- avg_fluo
- avg_dep
- dist_land
- dist_200m
- year

So in the interest of parsimony, we'll remove non-significant variables to create a new model:

```
bfal_poisson <- glm(bfal~avg_sal+avg_fluo+avg_dep+dist_land+dist_200m+year, data=transJul,family=poisson)
summary(bfal_poisson)
```

```
##
## Call:
## glm(formula = bfal ~ avg_sal + avg_fluo + avg_dep + dist_land +
##     dist_200m + year, family = poisson, data = transJul)
##
## Deviance Residuals:
##     Min       1Q   Median       3Q      Max
## -1.1781  -0.4238  -0.1848  -0.0421   5.5779
##
## Coefficients:
##               Estimate Std. Error z value Pr(>|z|)
## (Intercept) -5.029e+02  1.696e+02  -2.965  0.00303 **
```

```
## avg_sal      1.360e+00  6.582e-01   2.066  0.03884 *
## avg_fluo    -1.623e-01  6.147e-02  -2.640  0.00828 **
## avg_dep     -2.318e-03  4.833e-04  -4.796 1.62e-06 ***
## dist_land   -5.793e-05  3.228e-05  -1.795  0.07269 .
## dist_200m   -3.266e-04  6.200e-05  -5.268 1.38e-07 ***
## year         2.283e-01  8.047e-02   2.838  0.00455 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for poisson family taken to be 1)
##
##     Null deviance: 412.03  on 808  degrees of freedom
## Residual deviance: 298.70  on 802  degrees of freedom
## AIC: 406.11
##
## Number of Fisher Scoring iterations: 7
```

So from this we should be able to predict the spatial distribution of counts using the formula for the poisson glm model, which in our case will have 5 explanatory variables avg_sal (X_1), avg_fluo (X_2), avg_dep (X_3), dist_land (X_4), dist_200m (X_5), and year ($X_6), using the coefficient estimates from the summary above and the prediction formula in the form:

$$z = e^{b_0+b_1X_1+b_2X_2+b_3X_3+b_4X_4+b_5X_5+b_6X_6}$$

7.3.1 Map the prediction

To create a map of the predicted spatial pattern, we'll need rasters for each of the variables. We'll need to create these in various ways:

- Depth raster from bathymetry data source
- Derive distance rasters from the mainland and 200 m isobar features
- For measurements obtained on the transect cruises like those from the CTD sensor (salinity and temperature), we'll need to interpolate a raster from those points

7.3.1.1 Depth and distance rasters

We have a depth raster at 200-m resolution, but for our purposes and at this scale we only need 1000-m (1-km) resolution. We'll create a 1-km template raster based on a 10 km buffer around data points, and use that to resample the distances and along with mainland and 200-m isobath, derive the distance rasters needed:

```
library(terra)
bathy <- project(rast(ex("SFmarine/bd200m_v2i.tif")),crs(transects))
```

```
AOI <- vect(st_union(st_buffer(transJul, 10000)))
rasAOI <- rast(AOI, res=1000)
bathy1km <- terra::resample(bathy,rasAOI)
distland <- terra::distance(rasAOI, vect(mainland))
```

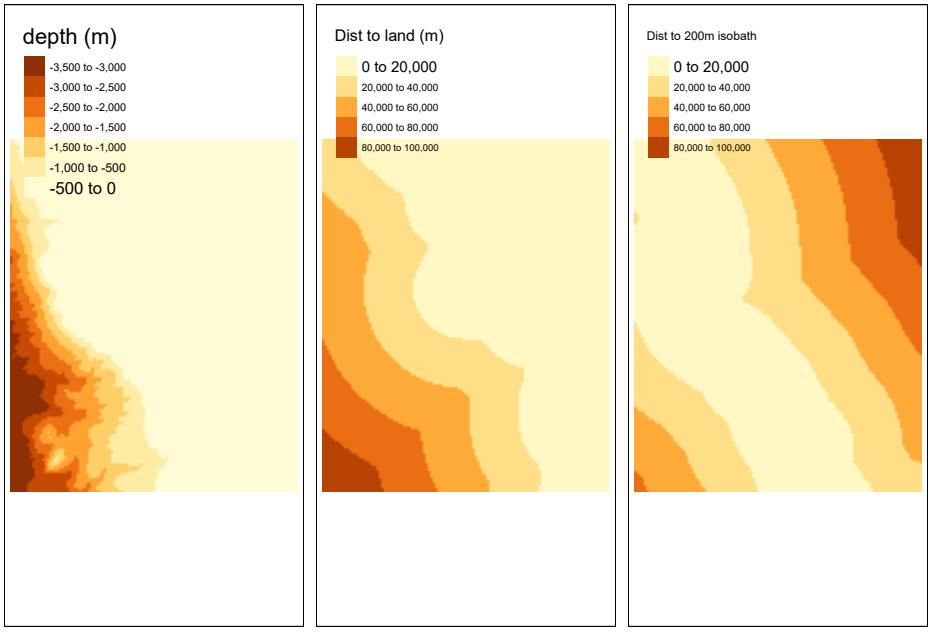
```
## |---------|---------|---------|---------|=========================================
```

```
names(distland) <- "distLand"
distisobath200 <- terra::distance(rasAOI, vect(isobath_200))
```

```
## |---------|---------|---------|---------|=======================================
```

```
names(distisobath200) <- "dist200mD"
```

```
tm_dep <- tm_shape(bathy1km) + tm_raster(title="depth (m)")
tm_distland <- tm_shape(distland) + tm_raster(title="Dist to land (m)")
tm_distisobath200 <- tm_shape(distisobath200) + tm_raster(title="Dist to 200m isobath")
tmap_arrange(tm_dep, tm_distland, tm_distisobath200)
```



7.4 Interpolation

We'll use gstat with an inverse distance weighted (IDW) method applying a inverse distance power of 2 for temperature, salinity and fluorescence, for each year `2004:2013`, then average the 10.

```
library(gstat)
transJulv <- vect(transJul)
d <- data.frame(geom(transJulv)[,c("x", "y")], as.data.frame(transJulv))
   # d is a data.frame with x and y variables from the geom
# Temperature
for (year in 2004:2013) {
  gs <- gstat(formula=avg_tem~1, locations=~x+y, data=d, set=list(idp=2))
  temRas <- interpolate(rasAOI, gs)$var1.pred
  if (year == 2004) temSum <- temRas
  if (year > 2004) temSum <- temSum + temRas
}
temMean <- temSum/10; fillval <- global(temMean, "mean")[,1]
```

```
tem <- focal(temMean,9, "mean", fillvalue=fillval)
names(tem) <- "temperature"
# Salinity
for (year in 2004:2013) {
  gs <- gstat(formula=avg_sal~1, locations=~x+y, data=d, set=list(idp=2))
  salRas <- interpolate(rasAOI, gs)$var1.pred
  if (year == 2004) salSum <- salRas
  if (year > 2004) salSum <- salSum + salRas
  #print(paste(paste(year, "salSum", global(salSum, "max")[,1])))
}
salMean <- salSum/10; fillval <- global(salMean, "mean")[,1]
sal <- focal(salMean,9, "mean", fillvalue=fillval) # generalizes the IDW to reduce sample point artifacts
names(sal) <- "salinity"
# Fluorescence
for (year in 2004:2013) {
  gs <- gstat(formula=avg_fluo~1, locations=~x+y, data=d, set=list(idp=2))
  fluoRas <- interpolate(rasAOI, gs)$var1.pred
  if (year == 2004) fluoSum <- fluoRas
  if (year > 2004) fluoSum <- fluoSum + fluoRas
}
fluoMean <- fluoSum/10; fillval <- global(fluoMean, "mean")[,1]
fluo <- focal(fluoMean,9, "mean", fillvalue=fillval) # generalizes the IDW to reduce sample point artifacts
names(fluo) <- "fluorescence"
```

```
tm_tem <- tm_shape(tem) + tm_raster(title="temperature")
tm_sal <- tm_shape(sal) + tm_raster(title="salinity")
tm_fluo <- tm_shape(fluo) + tm_raster(title="fluorescence")
tmap_arrange(tm_dep, tm_tem, tm_sal, tm_fluo)
```

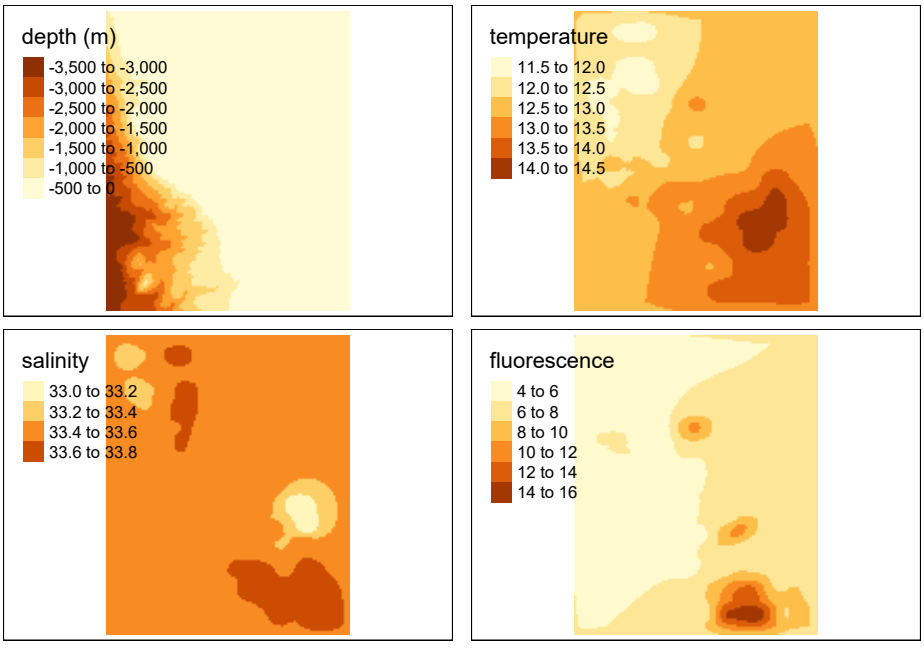


Figure 7.3 Interpolated depth, salinity and fluorescence, mean of July samples in years 2004:2011

Reviewing the results of the glm...

```
##
## Call:
## glm(formula = bfal ~ avg_sal + avg_fluo + avg_dep + dist_land +
##     dist_200m + year, family = poisson, data = transJul)
##
## Deviance Residuals:
##     Min       1Q   Median       3Q      Max
## -1.1781  -0.4238  -0.1848  -0.0421   5.5779
##
## Coefficients:
##               Estimate Std. Error z value Pr(>|z|)
## (Intercept) -5.029e+02  1.696e+02  -2.965  0.00303 **
## avg_sal      1.360e+00  6.582e-01   2.066  0.03884 *
## avg_fluo    -1.623e-01  6.147e-02  -2.640  0.00828 **
## avg_dep     -2.318e-03  4.833e-04  -4.796 1.62e-06 ***
## dist_land   -5.793e-05  3.228e-05  -1.795  0.07269 .
## dist_200m   -3.266e-04  6.200e-05  -5.268 1.38e-07 ***
## year         2.283e-01  8.047e-02   2.838  0.00455 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for poisson family taken to be 1)
##
##     Null deviance: 412.03  on 808  degrees of freedom
## Residual deviance: 298.70  on 802  degrees of freedom
## AIC: 406.11
##
## Number of Fisher Scoring iterations: 7
```

... we can set up and apply the prediction formula to these rasters, using year 2006.

```
b <- coefficients(bfal_poisson)
bfalpred <- exp(b[1] + b[2]*sal + b[3]*fluo + b[4]*bathy1km + b[5]*distland + b[6]*distisobath200 + b[7]*2006)
names(bfalpred) = "bfal_predict"
plot(bfalpred)
lines(vect(isobath_200), add=T, col="blue")
contour(bathy1km, add=T)
```

It appears that a prominent influence on the resulting spatial pattern is distance to the 200-m isobath, with details provided by the other explanatory variables. Given that the year is a constant in the spatial domain, the pattern on the prediction map does not vary, but the range of counts predicted varies, and we can predict the maximum value with:

```
cat("Maximum black-footed albatross model count predictions, 2004:2013\n")
```

```
## Maximum black-footed albatross model count predictions, 2004:2013
```

```
for (i in 2004:2013) {
  maxprd <- global(exp(b[1] + b[2]*salRas + b[3]*fluoRas + b[4]*bathy1km + b[5]*distland + b[6]*distisobath200 +
  cat(as.character(i), " ", as.character(format(maxprd, digits=3)), "\n")
}
```

```
## 2004   0.305
```

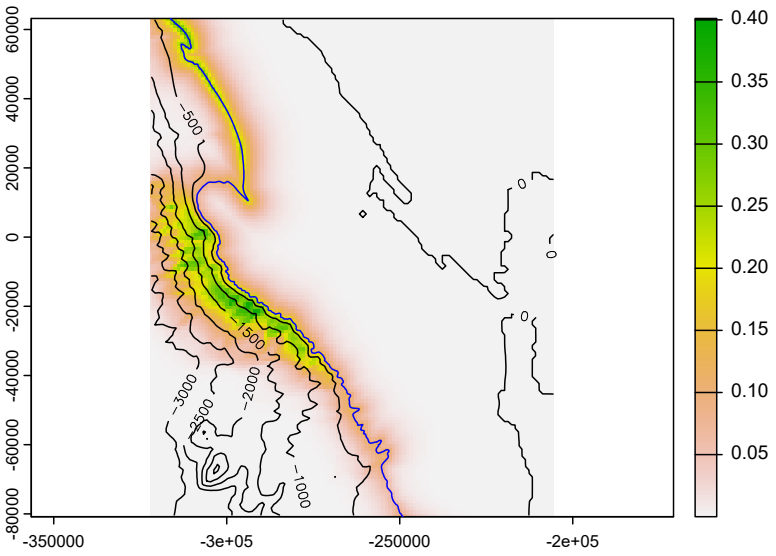


Figure 7.4 Black-footed albatross prediction, July 2006

```
## 2005   0.383
## 2006   0.481
## 2007   0.604
## 2008   0.759
## 2009   0.954
## 2010   1.2
## 2011   1.51
## 2012   1.89
## 2013   2.38
```

Chapter 8

Time Series case studies

8.1 Loney Meadow flux data

At the beginning of this chapter, we looked at an example of a time series in flux tower measurements of northern Sierra meadows, such as in Loney Meadow where during the 2016 season a flux tower was used to capture CO_2 flux and related micrometeorological data.

We also captured multispectral imagery using a drone, allowing for creating high-resolution (5-cm pixel) imagery of the meadow in false color (with NIR as red, red as green, green as blue), useful for showing healthy vegetation (as red) and water bodies (as black).

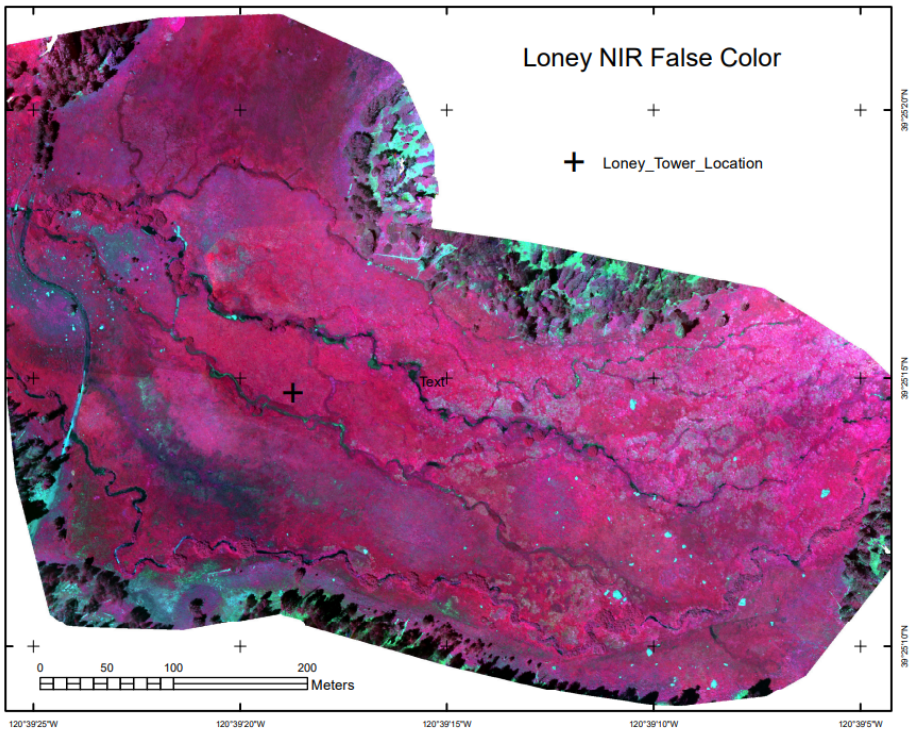


Figure 8.1 Loney Meadow False Color image from drone-mounted multispectral camera, 2017

The flux tower data were collected at a high frequency for eddy covariance processing where 3D wind speed

Figure 8.2 Flux tower installed at Loney Meadow, 2016. Photo credit: Darren Blackburn

data are used to model the movement of atmospheric gases, including CO_2 flux driven by photosynthesis and respiration processes. Note that the sign convention of CO_2 flux is that positive flux is release to the atmosphere, which might happen when less photosynthesis is happening but respiration and other CO_2 releases continue, while a negative flux might happen when more photosynthesis is capturing more CO_2.

A spreadsheet of 30-minute summaries from 17 May to 6 September can be found in the `igisci` extdata folder as `"meadows/LoneyMeadow_30minCO2fluxes_Geog604.xls"`, and includes data on photosynthetically active radiation (PAR), net radiation (Qnet), air temperature, relative humidity, soil temperature at 2 and 10 cm depth, wind direction, wind speed, rainfall, and soil volumetric water content (VWC). There's clearly a lot more we can do with these data (see Blackburn, Oliphant, and Davis (2021)), but we'll look at CO_2 flux and PAR using some of the same methods we've just explored.

First we'll test read in the data (I'd encourage you to also look at the spreadsheet in Excel [but don't change it] to see how it's organized) …

```
read_xls(ex("meadows/LoneyMeadow_30minCO2fluxes_Geog604.xls"))
```

```
## # A tibble: 5,398 x 14
##    Decim~1 Day o~2 Hour ~3 CO2 F~4 PAR   Qnet  Tair  RH    Tsoil~5 Tsoil~6 wdir
##      <dbl>   <dbl>   <dbl> <chr>   <chr> <chr> <chr> <chr> <chr>   <chr>   <chr>
##  1     NA      NA      NA   mgC m-~ mmol~ W m-2 degC  %     degC    degC    deg
##  2    138     138      24   0.2011~ 0     -93.~ 8.44~ 62.9~ 10.672~ 12.582~ 58.8~
##  3    138.    138       0.5 0.1096~ 0     -95.~ 8.12~ 64.5~ 10.364~ 12.441~ 48.8~
##  4    138.    138       1   0.1650~ 0     -94.~ 7.60~ 66.9~ 10.0718 12.300~ 37.7~
##  5    138.    138       1.5 0.1547~ 0     -96.~ 7.57~ 66.8~ 9.7869~ 12.159~ 42.9~
##  6    138.    138       2   0.1783~ 0     -97.~ 7.50~ 67.1~ 9.5242~ 12.018~ 49.5~
##  7    138.    138       2.5 0.1356~ 0     -99.~ 7.61~ 65.3~ 9.2895~ 11.879~ 70.9~
##  8    138.    138       3   0.1512~ 0     -101~ 7.73~ 63.7~ 9.0644~ 11.741~ 64.9~
##  9    138.    138       3.5 0.1698~ 0     -100~ 7.48~ 64.8~ 8.8473~ 11.604~ 71.9~
## 10    138.    138       4   0.1340~ 0     -98.~ 7.36~ 65.4~ 8.6596~ 11.470~ 59.5~
## # ... with 5,388 more rows, 3 more variables: wspd <chr>, Rain <chr>,
## #   VWC <chr>, and abbreviated variable names 1: `Decimal time`,
## #   2: `Day of Year`, 3: `Hour of Day`, 4: `CO2 Flux`, 5: `Tsoil 2cm`,
```

```
## #   6: `Tsoil 10cm`
```

… and see that just as with the Bugac and Hungary data, it has half-hour readings and the second line of the file has measurement units. There are multiple ways of dealing with that, but this time we'll capture the variable names then add them back after removing the first two rows:

```
library(igisci)
library(readxl); library(tidyverse); library(lubridate)
vnames <- read_xls(ex("meadows/LoneyMeadow_30minCO2fluxes_Geog604.xls"), n_max=0) %>%
    names()
vunits <- read_xls(ex("meadows/LoneyMeadow_30minCO2fluxes_Geog604.xls"), n_max=0, skip=1) %>%
    names()
Loney <- read_xls(ex("meadows/LoneyMeadow_30minCO2fluxes_Geog604.xls"), skip=2, col_names=vnames) %>%
  rename(DecimalDay = `Decimal time`,
         YDay = `Day of Year`,
         Hour = `Hour of Day`,
         CO2flux = `CO2 Flux`,
         Tsoil2cm = `Tsoil 2cm`,
         Tsoil10cm = `Tsoil 10cm`) %>%
  mutate(datetime = as_date(DecimalDay, origin="2015-12-31"))
```

The time unit we'll want to use for time series is going to be days, and we can also then look at the data over time, and a group_by-summarize process by days will give us a generalized picture of changes over the collection period reflecting phenological changes from first exposure after snowmelt through the maximum growth period and through the major senescence period of late summer. We'll look at a faceted graph from a pivot_long table.

```
LoneyDaily <- Loney %>%
  group_by(YDay) %>%
  summarize(CO2flux = mean(CO2flux),
            PAR = mean(PAR),
            Qnet = mean(Qnet),
            Tair = mean(Tair),
            RH = mean(RH),
            Tsoil2cm = mean(Tsoil2cm),
            Tsoil10cm = mean(Tsoil10cm),
            wdir = mean(wdir),
            wspd = mean(wspd),
            Rain = mean(Rain),
            VWC = mean(VWC))
LoneyDailyLong <- LoneyDaily %>%
  pivot_longer(cols = CO2flux:VWC,
               names_to="parameter",
               values_to="value") %>%
  filter(parameter %in% c("CO2flux", "Qnet", "Tair", "RH", "Tsoil10cm", "VWC"))
p <- ggplot(data = LoneyDailyLong, aes(x=YDay, y=value)) +
  geom_line()
p + facet_grid(parameter ~ ., scales = "free_y")
```

Now we'll build a time series for CO_2 for an 8-day period over the summer solstice, using the start time and frequency (there's also a time stamp, but this was easier, since I knew the data had no gaps):

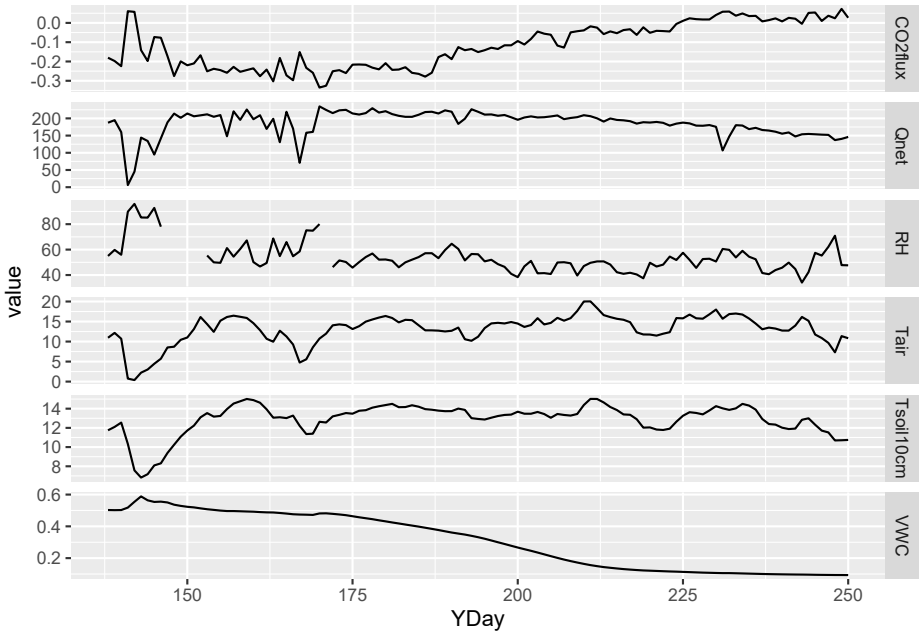


Figure 8.3 Facet plot with free y scale of Loney flux tower parameters

```
LoneyCO2 <- Loney %>%
  filter(YDay > 167 & YDay < 176) %>%
  dplyr::select(CO2flux) %>%
  ts(start=138,frequency=48)
plot(decompose(LoneyCO2))
```

Finally, we'll create a couple ensemble average plots from all of the data, with sd error bars similar to what we did for Manaus, and with cowplot used again to compare two ensemble plots:

```
library(cowplot)
LoneyCO2sum <- Loney %>%
  group_by(Hour) %>%
  summarize(meanCO2 = mean(CO2flux), sdCO2 = sd(CO2flux))
LoneyPARsum <- Loney %>%
  group_by(Hour) %>%
  summarize(meanPAR = mean(PAR), sdPAR = sd(PAR))
p1 <- LoneyCO2sum %>% ggplot(aes(x=Hour)) + scale_x_continuous(breaks=seq(0,24,3)) +
  geom_line(aes(y=meanCO2), col="blue") +
  geom_errorbar(aes(ymax = meanCO2 + sdCO2, ymin = meanCO2 - sdCO2)) +
  ggtitle("Loney ensemble CO2 flux")
p2 <-  LoneyPARsum %>% ggplot(aes(x=Hour)) + scale_x_continuous(breaks=seq(0,24,3)) +
  geom_line(aes(y=meanPAR), col="red") +
  geom_errorbar(aes(ymax = meanPAR + sdPAR, ymin = meanPAR - sdPAR)) +
  ggtitle("Loney ensemble PAR")
plot_grid(plotlist=list(p1,p2), ncol=1, align='v') # from cowplot
```

We can explore the Loney meadow data further, maybe comparing multiple ensemble averages, relating variables (like the example here):

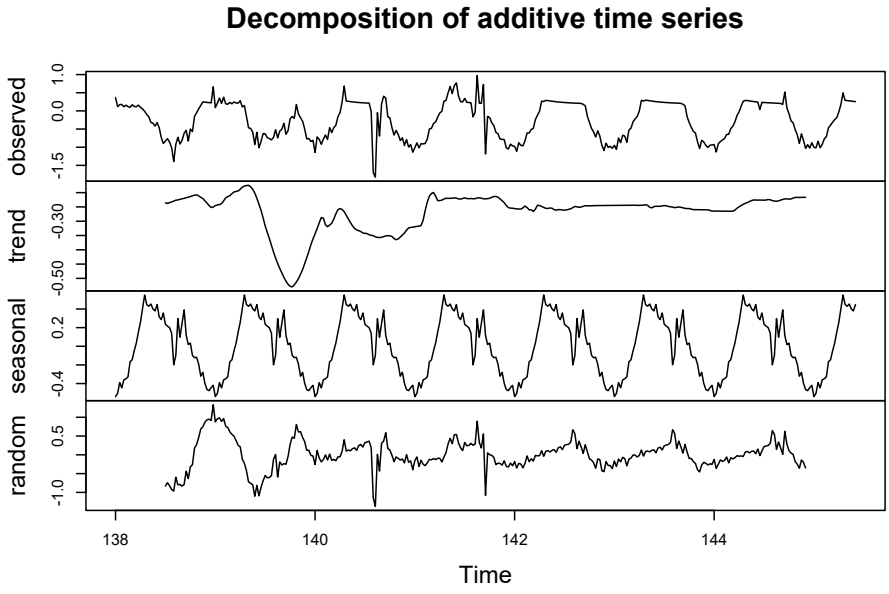


Figure 8.4 Loney CO 2 decomposition by day, 8-day period at summer solstice

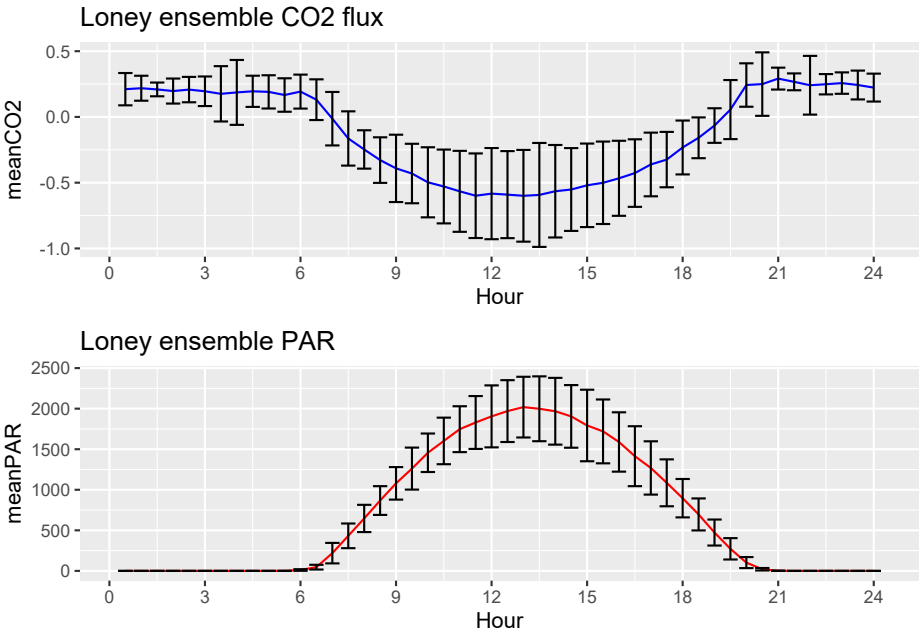


Figure 8.5 Loney meadow CO 2 and PAR ensemble averages

```
library(igisci)
library(readxl); library(tidyverse); library(lubridate)
vnames <- read_xls(ex("meadows/LoneyMeadow_30minCO2fluxes_Geog604.xls"), n_max=0) %>%
    names()
vunits <- read_xls(ex("meadows/LoneyMeadow_30minCO2fluxes_Geog604.xls"), n_max=0, skip=1) %>%
    names()
Loney <- read_xls(ex("meadows/LoneyMeadow_30minCO2fluxes_Geog604.xls"), skip=2, col_names=vnames) %>%
  rename(DecimalDay = `Decimal time`,
         YDay = `Day of Year`,
         Hour = `Hour of Day`,
         CO2flux = `CO2 Flux`,
         Tsoil2cm = `Tsoil 2cm`,
         Tsoil10cm = `Tsoil 10cm`) %>%
  mutate(datetime = as_date(DecimalDay, origin="2015-12-31"))
ggplot(data=Loney) + geom_point(aes(x=Qnet, y=CO2flux, col=YDay), alpha=0.5) +
   scale_color_gradient2(low="blue", high="red", mid="green", midpoint=200)
```

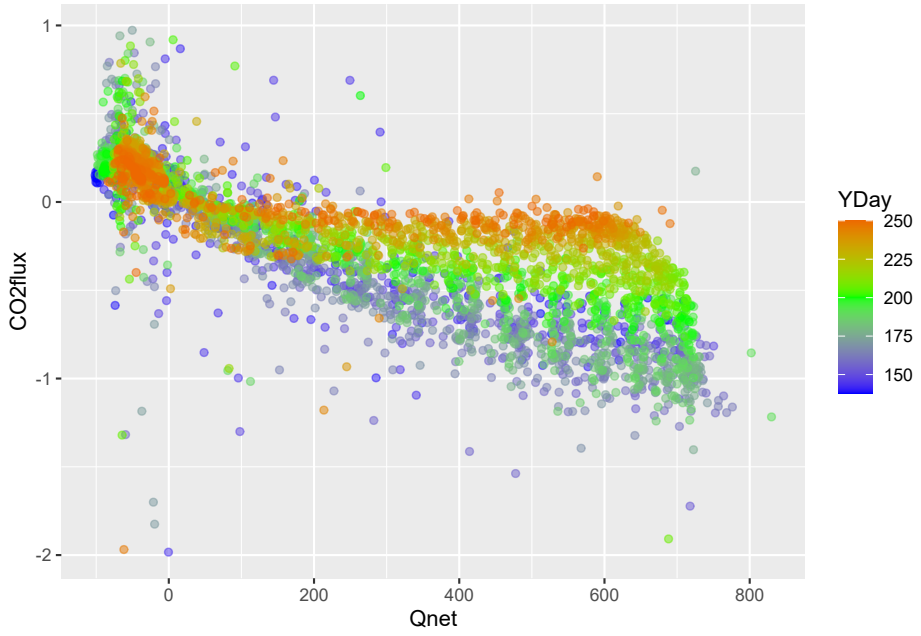


Figure 8.6 Loney CO2 flux vs Qnet

Chapter 9

R Markdown and Bookdown

This book is built using R Markdown and bookdown methods, and published at http://bookdown.org. There are some more comprehensive resources on both of these at Xie, Allaire, and Grolemund (2019) and Xie (2021), and also the markdown cheat sheet at https://www.rstudio.com/wp-content/uploads/2015/02/rmarkdown-cheatsheet.pdf but I thought it might be useful to provide some things I've discovered.

- The first section is about R Markdown, and has the basics for creating a report with Markdown text and code outputs. To keep it simple, you might just go with html output.
- Next is a template that provides the basic settings that should work to create a report in either HTML, pdf, or Word formats, using the `tinytex` LaTeX interpreter you'll need to install if you want to create pdf output.
- The next section is more on bookdown extensions to R markdown, including how to build a book with chapters with a referencing system for figures with headings and text citations, indices, and other features.

- The pdf section describes more about the process for creating pdf via a LaTeX interpreter, with advanced stuff needed for controlling the format, especially if you want to go to a publisher who'll have very specific rules to follow.

9.1 R Markdown

Markdown isn't new and isn't at all restricted to R. It's the basis for Jupyter Notebooks, which I tend to use for Python but also works with R (the "py" in Jupyter is for Python and the "r" is for R); but I think R Markdown is better developed within RStudio, which provides more of an all-in-one *integrated development environment* (IDE).

For most users, what's covered in the cheat sheet, which should be at https://www.rstudio.com/wp-content/uploads/2015/02/rmarkdown-cheatsheet.pdf is all you need to be able to format nice R Markdown documents, but the more thorough treatment at Xie, Allaire, and Grolemund (2019) is a good resource for going further.

9.1.1 Markdown editing

The most basic thing to understand is that an R Markdown `.Rmd` document is divided into three elements:

- Optional header information at the top, marked off by --- in the first line, followed by any YAML settings you want to set, and closing with another line of ---.

- Markdown sections not delimited by anything, but responding to markdown formatting like pairs of `*` enclosing *text in italics*, pairs of `**` enclosing **bold text**, pairs of backticks enclosing `bits of code`, and lots of other things. Section headers are created with one or more hash `#` characters at the start of a line followed by the header text on that line, and when editing will show up as blue text like the italic and bold text; the top level has a single `#`, the next has `##` (like this **R Markdown** section), and so on. You can create bulleted lists with each item starting with a dash - after a blank line to get it started (see below), and lots of other things – see Xie, Allaire, and Grolemund (2019) and other sources for the multitude of formatting structures that markdown provides. Here's some of the text shown above as you enter it as markdown, though RStudio will wrap the long lines of text, and also provide color coding:

```
### Markdown editing
The most basic thing to understand is that an R Markdown `.Rmd` document is divided into three elements:

- Optional header information at the top, marked off by `---` in the first line, followed by any YAML settings yo
- Markdown sections not delimited by anything, but responding to markdown formatting like pairs of `*` enclosing
```

- Code chunks delimited by triple backticks, the first followed on its line with the code chunk header with `{r ...}` identifying R as the language along with other settings. A code chunk typically produces just one output (or maybe none at all, if it's just processing data you might use later). Here's a simple example, with one code chunk label and one setting, then code that creates and then displays a vector:

```
```{r chunklabel, include=FALSE}
a <- c(1, 7, 5)
a
```
```

```
## [1] 1 7 5
```

9.1.2 Display options in code chunks

The display options for code chunk headers `{r ...}` are very important to figure out in order to avoid unwanted messages in the rmarkdown document or book, and get figure headings working. Each option is separated by a comma, and follow the code chunk label. See https://www.rstudio.com/wp-content/uploads/2015/02/rmarkdown-cheatsheet.pdf for a list of these options. I've found that some of what's documented at https://yihui.org/knitr/options/#code-evaluation doesn't work for me, so here's what I've observed: `echo=TRUE` won't display the code if `include=FALSE` is set, though the above source suggests that the `include` option only refers to chunk output.

Display options:

1. no code no outputs : `include=F` [normal for exercises in book]
2. code no outputs : `include=T,fig.show="hide",results=F` (`include=T` is default) `results="markup"` is default (same as `TRUE`?)
3. code and outputs : `include=T,echo=T,fig.show=T,results=T` (return to defaults)
4. no code or no eval : `include=F,eval=F`

And there are lots of other code options, like including a numbered figure caption with `fig.cap="my figure caption"`. The bookdown output extensions add citing and linking to the figure in the text, using the referencing system.

Warnings and messages Turning off warnings and messages is so important in a book that I use `knitr::opts_chunk$set` to set these as defaults at the top of each chapter .Rmd.

```
```{r echo=F}
knitr::opts_chunk$set(echo=T, warning=F, message=F, fig.align='center', out.width="75%")
```
```

However some things that seem like messages are actually *results*, for instance:

`st_read` will create unwanted messages that are actually results, so you can either put it in a code chunk that isn't displayed at all with `include=F` or probably better use `results = "hide"`. For instance, the following code is in a code chunk that has three options `{r warning=F, results="hide", message=F}` to avoid printing quite a lot of unwanted stuff.

```
```{r warning=F, results="hide", message=F}
library(sf); library(igisci)
places <- st_read(ex("sierra/CA_places.shp"))
```
```

> You could of course have used that `knitr::opts_chunk$set` at the top of your code to avoid the warnings and messages, but you often want to see *results*, so hiding them is best isolated in a code chunk that isn't producing any results you want to see, and this is typically things like `st_read` I've found.

9.1.3 Numbered figures with text citations

Most of the your figures are going to be created by code in an individual code chunk, and it's a good idea to create a figure heading. You should organize your code chunk so that it only produces one figure, so you can use the code chunk header to provide the figure heading, and be able to cite the figure in text just before. *Note that the referencing capability requires bookdown output extensions to R Markdown, described below.*

To do this, once you've created a code chunk with an informative but unique chunk label (chunk labels cannot be reused in the document) like "visGraph", and also provide a figure caption with `fig.cap="..."` in the code chunk header, then in the text just above, you can use the referencing function `\@ref()` to cite the figure using the `fig:` key followed by the chunk label, either including parentheses or not:

- `Figure \@ref(fig:visGraph)` which produces this citation: Figure 9.1, or
- `(Figure \@ref(fig:visGraph))` which produces this citation: (Figure 9.1).

Here's what the code chunk and figure would then look like:

```
```{r visGraph, fig.cap="Vegetation bar graph"}
library(ggplot2); library(igisci)
ggplot(XSptsNDVI, aes(vegetation)) +
  geom_bar()
```
```

Or you may want to create a figure from an existing image, like a screen shot or picture, cited as usual as a figure reference `Figure \@ref(fig:TransectBuffersGoal)` (which produces this citation: Figure 9.2) and associated code chunk that uses the `knitr::include_graphics` method:

```
```{r TransectBuffersGoal,include=T,eval=T,echo=F,out.width='50%',fig.align="center",fig.cap="Transect Buffers (g
knitr::include_graphics(here::here("img","goal_spanlTransectBuffers.png"))
```
```

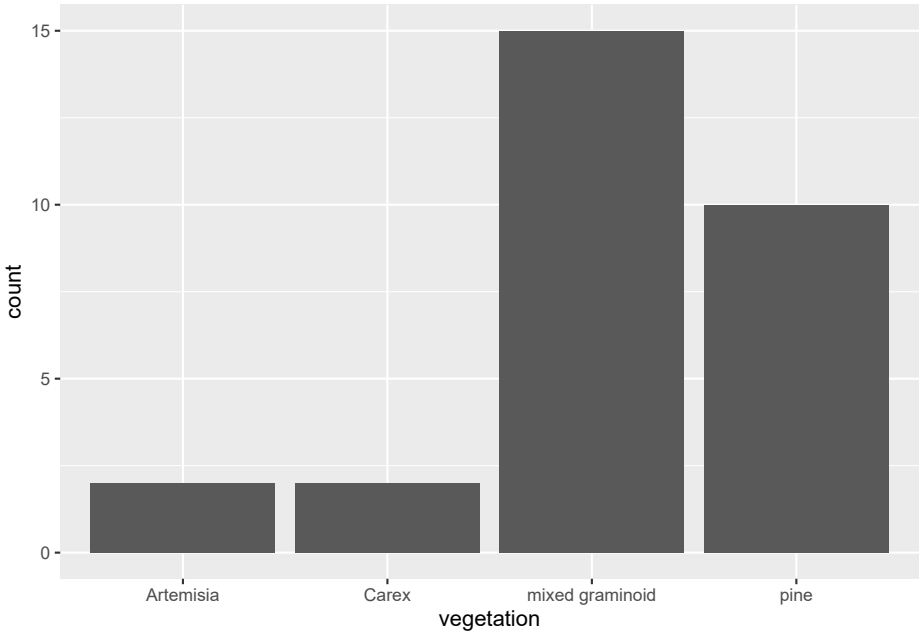


Figure 9.1 Vegetation bar graph

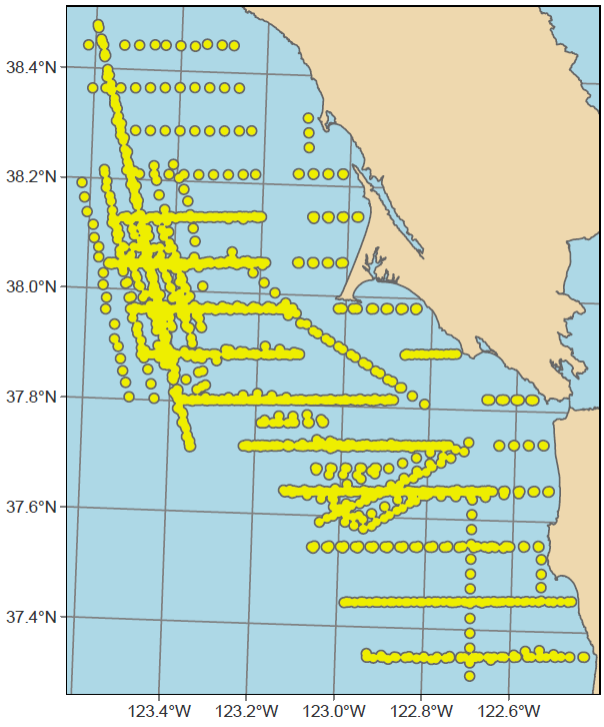


Figure 9.2 Transect Buffers (goal)

9.2 A template for multiple output formats

I've provided a basic template below that should work for a basic report that also uses some bookdown tricks such as citing figures using `\@ref()`. You'll need to have installed tinytex or other latex interpreter to use the pdf option. It also lets you choose to build HTML or Word outputs, as chosen by the `Knit` pull-down options in RStudio.

A pdf can be nice in working even when offline (though with no interactivity) and is great for creating a report to turn in to your boss or professor. As Yihui Xie notes in https://bookdown.org/yihui/bookdown/, however, you might want to stick with the simplicity and interactivity of HTML, as the effort of going to pdf might not be worth it. Creating a pdf does come with challenges, because the process involves two steps, each of which might create an error:

- RStudio creates a LaTeX .tex document
- Your LaTeX interpreter (such as tinytex) compiles the .tex to pdf.

Notes:

- While either HTML or Word (if you have Word installed) will automatically display their documents, the pdf choice just saves the pdf in your project folder where you need to open it with a pdf reader like Acrobat.
  - *You need to close that pdf before trying to create it again.*
- For pdf to work, you'll need to install a LaTeX interpreter like tinytex. See more information at https://yihui.org/tinytex/, but it's pretty straightforward, and can be done from RStudio:

```
install.packages('tinytex')
tinytex::install_tinytex()
```

9.2.1 The template

Copy and paste the following into a blank .Rmd file. Using the Knit button, try it out as is for the different formats, and then modify to provide your own markdown and code, while editing the header carefully. In the code chunk with the `knitr::opts_chunk$set` function, the `out.width="66%"` setting I chose is just something that tends to put more than one figure on a page. You might try higher values to get larger figures, or override that setting in a specific figure code chunk by including that parameter setting in its code chunk header.

````
---
title: "My Report"
author: "Jerry Davis"
date: "2022-12-30"
output:
  bookdown::pdf_document2:
    latex_engine: xelatex
  bookdown::html_document2: default
  bookdown::word_document2:
    toc: true
---

```{r echo=FALSE}
knitr::opts_chunk$set(echo=TRUE,warning=FALSE,message=FALSE,
````
````

````
                      fig.align='center',out.width="66%")
```
# My Report

Some built-in data, such as Old Faithful eruptions...
```{r}
head(faithful)
```

... and a time series of river flows of the Nile:
```{r}
Nile
```

## Figures

Here's a graph of CO2 concentrations recorded at Mauna Loa (Figure \@ref(fig:co2plot)), then a graph of Old Faith

```{r co2plot, fig.cap="CO2 time series at Mauna Loa"}
plot(co2)
```

```{r faithfulplot, fig.cap="Old Faithful eruptions"}
plot(faithful)
```
````

9.3 Building a book with bookdown and YAML options

While you can build a nice looking document in R Markdown, there are some other things that are useful for building a larger book out of it, like handling chapters, reference citations, and other features, so you'll want to review Xie (2021) to learn more. We've already seen that bookdown output formats provide capabilities like figure headings and citations.

The various files for your book can be hosted as a GitHub repository (more on GitHub in the next Addendum), and published to bookdown.org to share with others. You'll want to spend some time getting it well organized for others to access, and so we will want to learn more about YAML settings to set up our book format the way we want it.

We've already looked at YAML options in the header of our .Rmd file, and for a simple report that's all we need. However, with a more complex document we might call a book, it's good practice to organize YAML settings into various places:

- The header to the `index.Rmd` file that starts our book off, and is typically the first chapter, which will become `index.html` following standard web practice.
- `_output.yml`: which contains the settings for each of our output formats, what in our template we just put in our `.Rmd` header.
- `_bookdown.yml`: where we put the bookdown-specific settings, like the order of our chapters.

9.3.1 Building the book

What we've been doing with a single .Rmd file is Knitting it to various formats, however building a book is more complex and involves more files, including chapters, bibliographic references, and other things. Here are some notes:
````

- Use the Build menu in RStudio to specify whether you want to build which type you've specified in the `_output.yml` file. Since building a long book like this one can take a long time (about 35 minutes for both html and pdf for this book), I sometimes just build the HTML version, then build the pdf separately if it works, and since most people will use the HTML version I update that more frequently.
  - Note that pdfs don't include interactive maps except as a static rendering.
- Learn how to organize the book with chapters set aside by `#` in markdown then subheadings at various levels using `##`, `###`, etc., and don't forget the space after the last hash.
- The automatic numbering of chapters and subheadings is useful, but if you want a section to not use numbering, like this appendix, including `{-}` at the end of the unnumbered section header line.
- Creating sections, like **Spatial**, is done by including '(PART) before the section name, like the following, which starts off the section with this special heading followed by the first chapter:

```
# (PART) Spatial {-}

# Spatial Data and Maps

```
```

Note that the section is excluded from the automatic numbering with `{-}`.

I guess I could have included some text to introduce the section but the above works to include the section name in the table of contents.

9.3.2 Special characters and formatting limitations and challenges

Handling special characters and formatting is often a challenge for automated systems that try to build graphics with nicely formatted text from an array of inputs. Code chunk headers allow another level of automation, allowing us to produce figure headings and cite them in the text. In formatting text from markdown sections, we need to turn plain text into nicely formatted text, and markdown has a lot of ways of doing this. However all of this automation runs into limitations, since we run into coding-related rules with multiple systems interacting, and this is especially true when we add LaTeX rules to things. So we need to be aware of some often surprising limitations, such as:

- Code chunk labels can't have underscores and other special characters like `&` and `_`.
- Subscripts and superscripts can be confusing between html and LaTeX. See https://www.math.mcgill.ca/yyang/regression/RMarkdown/example.html .using ~ and ^ like CO_2 and X^2 might work in Rmarkdown and html, but not in LaTeX/pdf.

9.4 YAML files I used to create the Introduction to Environmental Data Science book.

Here are some important things I've discovered in building the book. The YAML files _output.yml, _bookdown.yml, and the index.Rmd need to be set up right, and it's difficult to know what goes in which.

However, *don't use these as a model*, unless you're going to a book publisher, and even then the settings will likely differ. But it illustrates some of the things you can do with YAML files.
```

9.4.1 _output.yml for EnvDataSci

Note that there are two output sections, one the `gitbook` style of HTML which provides a table of contents panel on the left, along with the cover art, the other the `pdf_book` section, which was the most challenging, and required working with the publisher to get the specific settings right. Building something as complex as a book often requires some tricky debugging, and since you're dealing potentially with R, Markdown, and LaTeX issues, can take a lot of effort, and if you need the format to work in a specific way (e.g. the way a publisher might require it), I needed to learn more about LaTeX, and you can find some tricks below.

```
bookdown::gitbook:
  css: style.css
  config:
    toc:
      collapse: none
      before: |
        <li><img src="img/eaRth_anna296.png"></li>
      after: |
     <li><a href="https://github.com/rstudio/bookdown" target="blank">Published with bookdown</a></li>
    book_filename: "envdatasci"
  toc_depth: 4
bookdown::pdf_book:
  includes:
    in_header: latex/preamble.tex
    before_body: latex/before_body.tex
    after_body: latex/after_body.tex
  keep_tex: true
  dev: "cairo_pdf"
  latex_engine: xelatex
  template: null
  pandoc_args: --top-level-division=chapter
  toc_depth: 3
  toc_unnumbered: false
  toc_appendix: true
  quote_footer: ["\\VA{", "}{}"]
  highlight_bw: true
```

- note the latex_engine: xelatex – that required specifying the xelatex engine, which you can set in RStudio Build/Configure Build Tools in the Sweave section of the Project Options window that will display and choosing XeLaTeX. For CRC, I needed the features this has, but didn't realize it until after a *lot* of trial & error.

9.4.2 index.Rmd for EnvDataSci

These settings are a higher level than what's in _output.yml, though you could put the things in that file here under an `output:` line. See examples in https://bookdown.org/yihui/bookdown/. *And, again, you're not going to want to use this as a model, since it has some specific settings for CRC Press.*

```
---
title: "Introduction to Environmental Data Science"
author: "Jerry Davis, SFSU Institute for Geographic Information Science"
date: "2022-12-30"
documentclass: krantz
```

```
classoption: krantz2
bibliography: [book.bib]
biblio-style: apalike
link-citations: yes
colorlinks: yes
lot: no
lof: yes
site: bookdown::bookdown_site
description: Background, methods and exercises for using R for environmental data
  science.  The focus is on applying the R language and various libraries for data
  abstraction, transformation, data analysis, spatial data/mapping, statistical modeling,
  and time series, applied to environmental research. Applies exploratory data analysis
  methods and tidyverse approaches in R, and includes contributed chapters presenting
  research applications, with associated data and code packages.
github-repo: iGISc/EnvDataSci
graphics: yes
geometry: margin=0.75in
---
```

This includes krantz to specify the krantz.cls file provided by CRC Press, the publisher I'm going with for the main book, and why I'm going to the major trouble of getting the LaTeX working. The classoption:krantz2 ends up creating the following in the .tex file, which is needed by the publisher to get the page size right for CRC. The krantz.cls file used was edited for me by the original programmer who wrote it. The classoption: krantz2 then produces the following in the .tex document.

```
\documentclass[
  krantz2]{krantz}
```

9.4.3 _bookdown.yml for EnvDataSci

The main thing I use this for is the list of chapters, which is nice to specify instead of letting the RStudio figure out the chapters based on their names by using numbers like 01_introduction, etc., and I can comment out parts – so I've set them up one line at a time to make it easy to use Ctrl-Sh-C to toggle commenting.

```
book_filename: "EnvDataSci"
clean: [packages.bib, bookdown.bbl]
delete_merged_file: true
rmd_files: ["index.Rmd",
            "introduction.Rmd",
            "abstraction.Rmd",
            "visualization.Rmd",
            "transformation.Rmd",
            "spatialDataMaps.Rmd",
            "spatialAnalysis.Rmd",
            "spatialRaster.Rmd",
            "spatialInterpolation.Rmd",
            "statistics.Rmd",
            "modeling.Rmd",
            "ImageryAnalysisClassification.Rmd",
            "TimeSeries.Rmd",
            "communication.Rmd",
            "references.Rmd"]
```

```
language:
  label:
    fig: "FIGURE "
    tab: "TABLE "
  ui:
    edit: "Edit"
    chapter_name: "Chapter "
```

9.4.3.1 Publishing

To get it published at bookdown.org, you'll need an account at Posit Connect, and then replacing "myAccount" with your account name, use:

```
bookdown::publish_book(account="myAccount", server="bookdown.org")
```

the first time, then after that you don't need those two parameters.

Chapter 10

Building a Package on GitHub for Data and Code

These are just some notes mostly on building packages, based mostly on a couple of great sources which you should use to learn the process:

- *R Packages* https://r-pkgs.org/ (Wickham and Bryan (n.d.))
  - Includes a quick run through in chapter 2 for building a package, with a function
  - Chapter 14 "External data" goes over how to build data into the package
  - Various chapter references below refer to this source
- *Happy Git and GitHub for the useR* https://happygitwithr.com/ (Bryan (n.d.))
  - All about how to get Git and GitHub working

Git and GitHub

You'll need to learn about using Git methods to host a package on GitHub (https://github.com), where you'll also need at least a free account. See *Happy Git and GitHub for the useR* https://happygitwithr.com/ (Bryan (n.d.)) to learn everything you'll need for this. Some key takeaways:

- The purpose of Git is version control, and https://github.com is a place to host package repositories (repos) that use this. Methods include keeping track of all changes in your package, including all files in the folders.
- You'll need to use GitHub to create an account and connect with collaborators. And you can use it for setting everything up in your package repository.
- You'll need to have the same folders on your local computer, and a method for *committing* and *pushing* up changes to GitHub.
  - You can use RStudio for most of this, and you'll be making all of your edits on this anyway. You'll see any changes showing up in the Git tab, where you can stage, commit, and push them to your repo.

  - GitHub Desktop can also do commits, pushes and pulls, and is very handy for updating changes to GitHub resulting from any package editing done in RStudio (or from any other changes to files and folders in the package folder, such as external data added to `extdata`). You might also find GitHub Desktop better for dealing with moving external files and folders manually stored in the extdata folder.

Some notes on the RStudio process

See Chapter 2 in *R Packages* (Wickham and Bryan (n.d.)) for a simple process. Some observations:

- You need `devtools` and `usethis` packages.
- You'll need to specify a folder with `create_package`, and you might want to put it in a `GitHub` folder in `Documents`. It doesn't matter where, but it's useful to keep GitHub things together. Refer to Chapter 2 https://r-pkgs.org/whole-game.html.
- This will create some essential files and folders:
  - `DESCRIPTION`: very important for naming and package information, which you'll be editing carefully
  - `NAMESPACE`: where functions are declared, generated by `roxygen2`
  - `R`: folder where your functions will go.
- Ultimately, other folders and files are created, and there's a diagram in https://r-pkgs.org/package-structure-state.html that illustrates the structure.
  - `man`: documentation on all of your functions and data sets
  - `inst`: various things go in here, but I've just used it for `extdata` where you can store .csv and other files
  - `data`: data (as .rda files) that can be called up like `co2` in the base R package.
- You might start by following the process in Chapter 2 for creating a simple package containing a simple function, using various `devtools` and `usethis` methods. This chapter also describes editing the `DESCRIPTION` file, using `roxygen2` to create the `man` file, how to document the function, and getting things ready to post to GitHub.
- On getting things ready for GitHub, it's very useful to use `check()` from time to time to see if there are any errors in the various setup files, like `DESCRIPTION`, file & folder structure, and documentation.
- The process can be confusing, but the various RStudio methods and functions provided in `devtools` and `usethis` make it at least very easy to do if you follow directions, which are spelled out pretty clearly in the *R Packages* book. We'll look at creating another function later in this appendix, following the method provided in *R Packages*

Data

For our package, `igisci`, we provided data in two ways:

- raw data as CSVs, shapefiles and TIFFs (useful since rasters don't seem to be supported in rda)
- `rda` files: normal external data that are ready to use as data frames and simple feature (sf) data

Raw data in extdata

Raw data (e.g. CSVs, shapefiles, and rasters) can be simply stored in the `inst/extdata` folder. Just create those folders and put the files there. Make sure to include all the files (like the multiple files that go with a shapefile). Then, to access the data once the data package is installed, the user just needs to use the `system.file()` function to provide the path and then use that with the appropriate read function; e.g. for a CSV, something like:

```
csvPath <- system.file("extdata","TRI/TRI_2017_CA.csv")
TRI_2017_CA <- read_csv(csvPath)
```

… or by using the `igisci::ex()` function, can be written as

```
TRI_2017_CA <- read.csv(ex("TRI/TRI_2017_CA.csv"))
```

… which isn't much different from reading data in your RStudio project folder, so seemed a useful method to include in this book. And adding data to the data package by simply putting them in the `extdata` folder is very easy and also works for data files like TIFFs that aren't supported as `rda` files that we'll look at next.

Binary data as `rda` files

For data needed frequently, similar to `co2` or other provided data in base R, you'll want to create binary data stored as `rda` files. These files need to be prepared from data in R and go in the data folder. The process is made very easy by using `usethis::use_data()` to add data as rda files to the `data` folder. These data can be data frames, simple features, and I'm sure other things.

I used a script `addData.R` that I put in the `inst` folder which built the data from some of the same files included in the extdata folder, with `usethis::use_data()` to store it in the data folder. Here's a simple example with just a csv converted directly, and it takes care of storing the result in the `data` folder as an `.rda` file:

```
sierraFeb <- read_csv("inst/extdata/sierra/sierraFeb.csv")
usethis::use_data(sierraFeb)
```

`devtools::document()`

This creates documentation on the data sets, using the file `R/data.R`, which will need to have lines of code similar to the following to document each data set. Note that the name of the data set goes last, in quotes. The formatting of the field names and descriptions is a bit tricky and doesn't follow normal R rules. As a result, sometimes my field names don't exactly match the actual field names. Maybe I'll get around to changing the original field names with `rename`. Note that the organization is important, with the title of the data first, a blank line, then a description, etc.:

```
#' Sierra February climate data
#'
#' Selection from SierraData to only include February data
#'
#' @format A data frame with 82 entries and 7 variables selected and renamed \describe{
#'   \item{STATION_NAME}{Station name}
#'   \item{COUNTY}{County Name}
#'   \item{ELEVATION}{Elevation in meters}
```

```
#'   \item{LATITUDE}{Latitude in decimal degrees}
#'   \item{LONGITUDE}{Longitude in decimal degrees}
#'   \item{PRECIPITATION}{February Average Precipitation in mm}
#'   \item{TEMPERATURE}{Febrary Average Temperature in degrees C}
#' }
#' @source \url{https://www.ncdc.noaa.gov/}
"sierraFeb"
```

Once these are on GitHub, a user can simply install the package with `devtools::install_github("iGISc/igisci")` – to use the `igisci` we created. Then to access the data just like built-in data, the user just needs to load that library with `library(igisci)`

10.0.0.1 Removing data

There may be a better way, but what worked was removing the `rda` and corresponding `man` files. Also edited the `data.R` file to remove them from the `man` file maybe being created again.

Code (functions)

You can build pretty extensive functions in a package, and you've been using these from a variety of packages. We'll just look at the simple example of a function we'll name `ex()` for making it easier to read data from the `extdata` folder of the `igisci` package. The methods shown here are based on https://r-pkgs.org/ which you should refer to for more thorough documentation.

Here's the function we'll create in the package. However, one caveat is that you want to add this after you've already installed the package with data, or you'll get an error.

```
ex <- function(dta){
  system.file("extdata",dta,package="igisci")
  }
```

We want to use it where in our code we wanted to read a file from `extdata` but we want cleaner code that doesn't take up much space in code we're writing than just reading it from your workspace. There's a csv file in the `sierra` folder of `extdata` named `Sierra2LassenData.csv` that we could then read with:

```
read.csv(ex("sierra/sierraFebShort.csv"))
```

```
##         STATION    COUNTY ELEVATION LATITUDE LONGITUDE PRECIPITATION TEMPERATURE
## 1     OROVILLE     Butte        52    39.52   -121.55           124        10.7
## 2       AUBURN    Placer       394    38.91   -121.08           160         9.7
## 3       SONORA  Tuolumne       511    37.97   -120.39           148         7.7
## 4  PLACERVILLE El Dorado       564    38.70   -120.82           171         9.2
## 5       COLFAX    Placer       725    39.09   -120.95           207         7.3
## 6  NEVADA CITY    Nevada       848    39.25   -121.00           268         6.7
## 7       QUINCY    Plumas      1042    39.94   -120.95           182         4.0
## 8     YOSEMITE  Mariposa      1225    37.75   -119.59           169         5.0
## 9      PORTOLA    Plumas      1478    39.81   -120.47            98         0.5
## 10     TRUCKEE    Nevada      1775    39.33   -120.17           126        -1.1
## 11  BRIDGEPORT      Mono      1972    38.26   -119.23            41        -2.2
## 12  LEE VINING      Mono      2072    37.96   -119.12            72         0.4
## 13       BODIE      Mono      2551    38.21   -119.01            40        -4.4
```

… and that doesn't take up much more code real estate than if you had a `"sierra/sierraFebShort.csv"` in your workspace/RStudio project folder. You just need to remember to also include `library(igisci)` in your code.

As you can see, this function is already in the igisci package, but how did it get there? Here are the steps for this simple function.

1. Use `usethis::use_r()` to create a script to hold your function(s). We'll just create one function, so we'll just name it the same name as the function, but you might want to have a set of maybe related functions in your script.

```
usethis::use_r("ex")
```

2. Add the function code to the `ex.R` script created.

```
ex <- function(dta){
  system.file("extdata",dta,package="igisci")
  }
```

3. Create the documentation skeleton while editing the script by using Insert Roxygen Skeleton from the RStudio Code menu.

4. Then fill in the various elements of the documentation (A title, parameters, what it returns and examples) which shows what you want the user to see with `?ex` or `?exfiles`, similar to this, and save the script. If you have more than one function, the documentation will go just above the function definition. See https://r-pkgs.org/man.html.

```
#' Access external data in package
#'
#' @param dta filename to access from extdata, including folders
#'
#' @return path to the file
#' @export
#'
#' @examples
#' read.csv(ex("sierra/sierraStations.csv"))
ex <- function(dta){
  system.file("extdata",dta,package="igisci")
}
#' Listing contents of extdata from package
#' @param NA
#' @return tibble
#' @export
#'
#' @examples
#' view(exfiles())
exfiles <- function(){
  library(dplyr)
  exfilesDF <- tribble(~dir,~file,~path,~type)
  for(d in list.dirs(ex(""),recursive=F)){
    #print(d)
    dsplit <- strsplit(d,"/")
    len <- length(dsplit[[1]])
```

```
    dir <- dsplit[[1]][len]
    for(shp in list.files(d,pattern="*.shp$")){
      exfilesDF <- bind_rows(exfilesDF,tribble(~dir,~file,~path,~type,dir,shp,paste0("ex(\'",dir,"/",shp,"\')"),"
    }
    for(tif in list.files(d,pattern="*.tif$")){
      exfilesDF <- bind_rows(exfilesDF,tribble(~dir,~file,~path,~type,dir,tif,paste0("ex(\'",dir,"/",tif,"\')"),"
    }
    for(csv in list.files(d,pattern="*.csv")){
      exfilesDF <- bind_rows(exfilesDF,tribble(~dir,~file,~path,~type,dir,csv,paste0("ex(\'",dir,"/",csv,"\')"),"
    }
    for(xls in list.files(d,pattern="*.xls")){
      exfilesDF <- bind_rows(exfilesDF,tribble(~dir,~file,~path,~type,dir,xls,paste0("ex(\'",dir,"/",xls,"\')"),"
    }
  }
  exfilesDF
}
```

5. Use `devtools::document()` to convert the comments to .Rd files.

```
devtools::document()
```

6. Then preview the documentation with `?ex`, and maybe use Check in the Build tab.
7. Then when you commit these changes in GitHub, the package will contain the function, ready to use as long as the user loads the library. You should see the function as `ex.Rd` in the `man` folder of your package, similar to what you see for the data described above.

This example was pretty simple for this simple code with no dependencies other than what's in base R, but you'll probably want to test it thoroughly, and add more documentation to do anything more complicated.

References

Applied California Current Ecosystem Studies. n.d. https://pointblue.org.

Ballard, Grant, Annie E Schmidt, Viola Toniolo, Sam Veloz, Dennis Jongsomjit, Kevin R Arrigo, and David G Ainley. 2019. "Fine-Scale Oceanographic Features Characterizing Successful Adélie Penguin Foraging in the SW Ross Sea." *Marine Ecology Progress Series*. https://doi.org/10.3354/meps12801.

Blackburn, Darren A, Andrew J Oliphant, and Jerry D Davis. 2021. "Carbon and Water Exchanges in a Mountain Meadow Ecosystem, Sierra Nevada, California." *Wetlands* 41 (3): 1–17. https://doi.org/10.1007/s13157-021-01437-2.

Bryan, Jenny. n.d. *Happy Git and GitHub for the useR*. https://happygitwithr.com/.

Davis, Jerry D, and George A Brook. 1993. "Geomorphology and Hydrology of Upper Sinking Cove, Cumberland Plateau, Tennessee." *Earth Surface Processes and Landforms* 18 (4): 339–62. https://doi.org/10.1002/esp.3290180404.

Studwell, Anna, Ellen Hines, Meredith L Elliott, Julie Howar, Barbara Holzman, Nadav Nur, and Jaime Jahncke. 2017. "Modeling Nonresident Seabird Foraging Distributions to Inform Ocean Zoning in Central California." *PLoS ONE*. https://doi.org/10.1371/journal.pone.0169517.

Wickham, Hadley, and Jenny Bryan. n.d. *R Packages*. O'Reilly. https://r-pkgs.org.

Xie, Yihui. 2021. *Bookdown: Authoring Books and Technical Documents with r Markdown*. Boca Raton, Florida: Chapman; Hall/CRC. https://bookdown.org/yihui/bookdown/.

Xie, Yihui, JJ Allaire, and Garrett Grolemund. 2019. *R Markdown: The Definitive Guide*. 1st ed. Boca Raton, Florida: Chapman; Hall/CRC. https://bookdown.org/yihui/rmarkdown/.

Index